



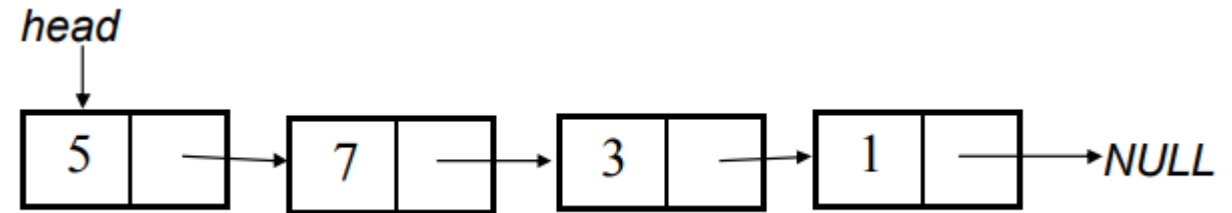
Linked List

Single Linked List

Single Linked List

- A linked list is a **linear collection** of data elements (called nodes), where the linear order is given by means of pointers.
- Each node is divided into 2 parts:
 - 1st part contains the information of the element.
 - 2nd part is called the link field or next pointer field which contains the address of the next node in the list. struct node

```
struct Node {  
    int value;  
    struct Node* nextNode;  
};
```

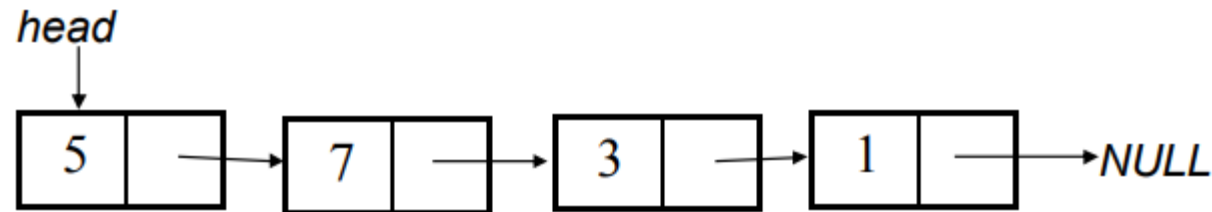


Single Linked List

- **Insert:** Add a new node in the **first**, **last** or **interior** of the list.
- **Delete:** Delete a node from the **first**, **last** or **interior** of the list.
- **Search:** Search a node containing particular value in the linked list.
- **Print** : Print the entire linked list

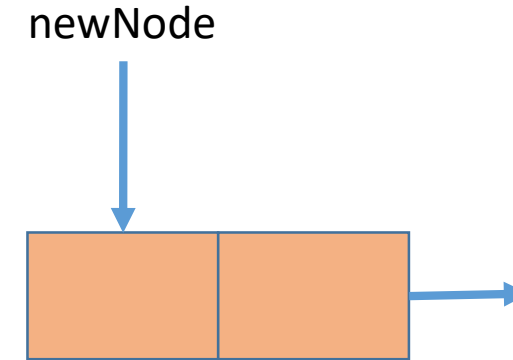
Linked List Insertion

- To add a new item or node, we at first need to make that node.
- Then it is to be inserted either at the beginning, or at the end, or after some particular value.
- **Before proceeding**, take a good look at the linked list we just saw.



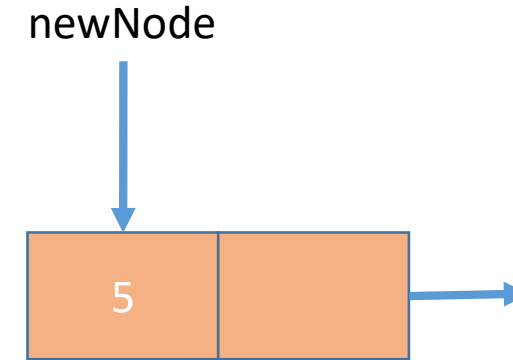
Linked List Insertion (At the beginning)

```
void addAtTheBeginning() {  
    // Making the new node  
    → struct Node* newNode = malloc(sizeof(struct Node));  
    printf("Enter the value of the new node\n");  
    scanf("%d", &newNode->value);  
    newNode->nextNode=NULL;  
  
    if(head==NULL) { // if the list is empty  
        head=newNode;  
    }  
    else { // if there are one or more nodes in the list  
        newNode->nextNode=head;  
        head=newNode;  
    }  
}
```



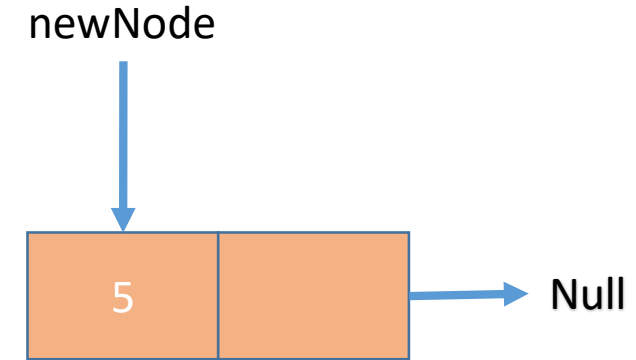
Linked List Insertion (At the beginning)

```
void addAtTheBeginning() {  
    // Making the new node  
    struct Node* newNode = malloc(sizeof(struct Node));  
    printf("Enter the value of the new node\n");  
    → scanf("%d", &newNode->value);  
    newNode->nextNode=NULL;  
  
    if(head==NULL) { // if the list is empty  
        head=newNode;  
    }  
    else { // if there are one or more nodes in the list  
        newNode->nextNode=head;  
        head=newNode;  
    }  
}
```



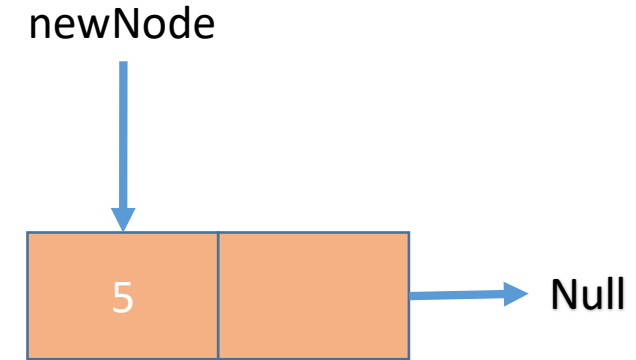
Linked List Insertion (At the beginning)

```
void addAtTheBeginning() {  
    // Making the new node  
    struct Node* newNode = malloc(sizeof(struct Node));  
    printf("Enter the value of the new node\n");  
    scanf("%d",&newNode->value);  
    → newNode->nextNode=NULL;  
  
    if(head==NULL) { // if the list is empty  
        head=newNode;  
    }  
    else { // if there are one or more nodes in the list  
        newNode->nextNode=head;  
        head=newNode;  
    }  
}
```



Linked List Insertion (At the beginning)

```
void addAtTheBeginning() {  
    // Making the new node  
    struct Node* newNode = malloc(sizeof(struct Node));  
    printf("Enter the value of the new node\n");  
    scanf("%d", &newNode->value);  
    newNode->nextNode=NULL;  
  
    → if (head==NULL) { // if the list is empty  
        head=newNode;  
    }  
    else { // if there are one or more nodes in the list  
        newNode->nextNode=head;  
        head=newNode;  
    }  
}
```

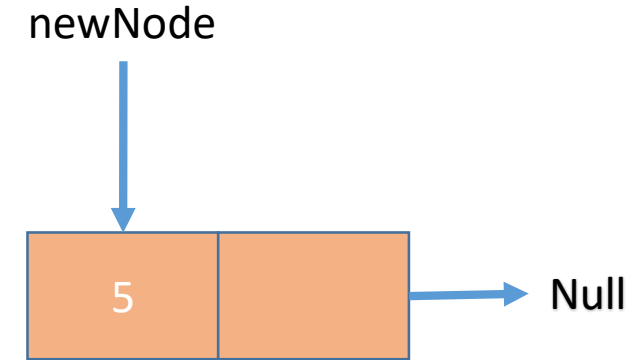


CASE 1: LIST IS EMPTY

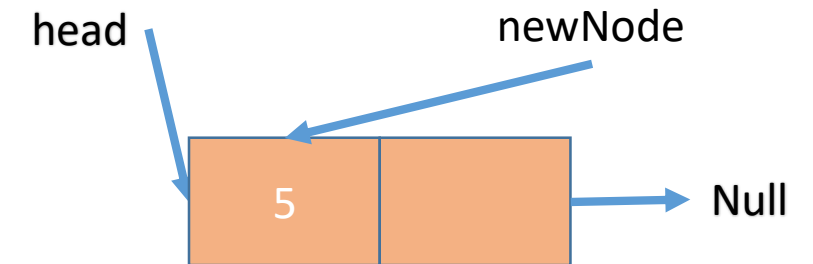


Linked List Insertion (At the beginning)

```
void addAtTheBeginning() {  
    // Making the new node  
    struct Node* newNode = malloc(sizeof(struct Node));  
    printf("Enter the value of the new node\n");  
    scanf("%d", &newNode->value);  
    newNode->nextNode = NULL;  
  
    if (head == NULL) { // if the list is empty  
        head = newNode;  
    } else { // if there are one or more nodes in the list  
        newNode->nextNode = head;  
        head = newNode;  
    }  
}
```



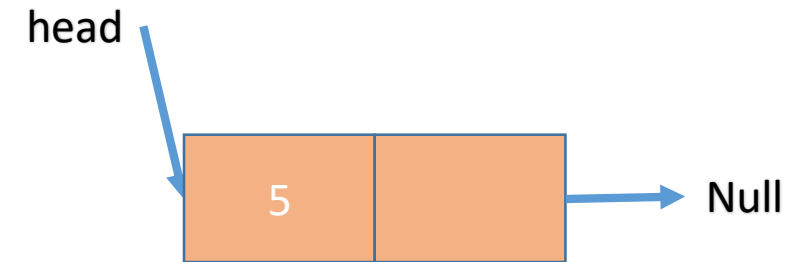
CASE 1: LIST IS EMPTY



Linked List Insertion (At the beginning)

CASE 2: LIST IS NO LONGER EMPTY

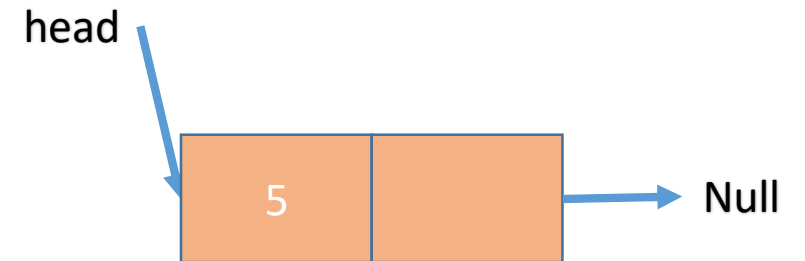
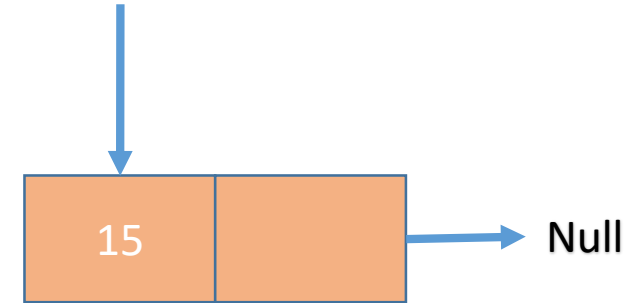
```
void addAtTheBeginning() {  
    → // Making the new node  
    struct Node* newNode = malloc(sizeof(struct Node));  
    printf("Enter the value of the new node\n");  
    scanf("%d",&newNode->value);  
    newNode->nextNode=NULL;  
  
    if(head==NULL) { // if the list is empty  
        head=newNode;  
    } else { // if there are one or more nodes in the list  
        newNode->nextNode=head;  
        head=newNode;  
    }  
}
```



Linked List Insertion (At the beginning)

```
void addAtTheBeginning() {  
    // Making the new node  
    struct Node* newNode = malloc(sizeof(struct Node));  
    printf("Enter the value of the new node\n");  
    scanf("%d", &newNode->value);  
    → newNode->nextNode=NULL;  
  
    if(head==NULL) { // if the list is empty  
        head=newNode;  
    } else { // if there are one or more nodes in the list  
        newNode->nextNode=head;  
        head=newNode;  
    }  
}
```

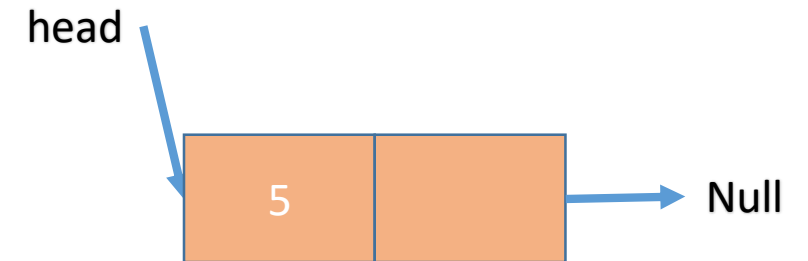
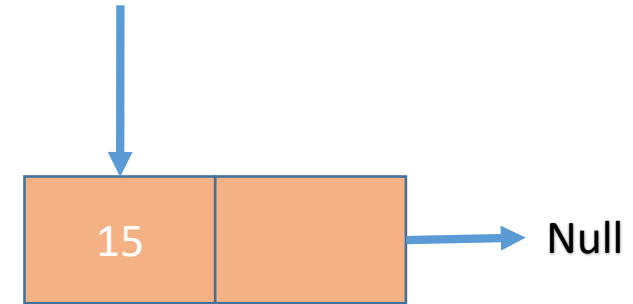
CASE 2: LIST IS NO LONGER EMPTY
newNode



Linked List Insertion (At the beginning)

```
void addAtTheBeginning() {  
    // Making the new node  
    struct Node* newNode = malloc(sizeof(struct Node));  
    printf("Enter the value of the new node\n");  
    scanf("%d", &newNode->value);  
    newNode->nextNode=NULL;  
  
    → if (head==NULL) { // if the list is empty  
        head=newNode;  
    }  
    else { // if there are one or more nodes in the list  
        newNode->nextNode=head;  
        head=newNode;  
    }  
}
```

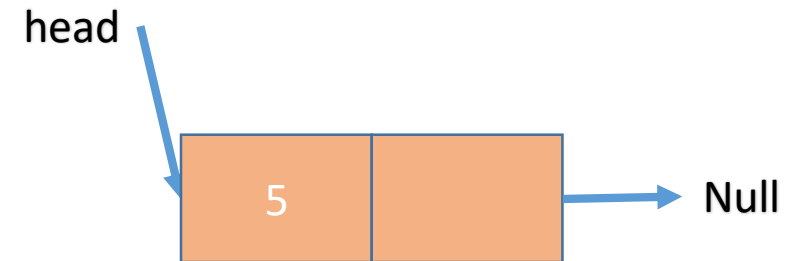
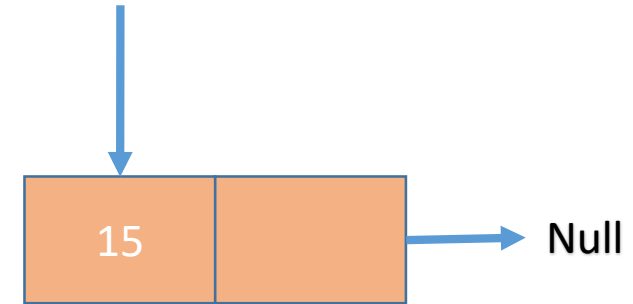
CASE 2: LIST IS NO LONGER EMPTY
newNode



Linked List Insertion (At the beginning)

```
void addAtTheBeginning() {  
    // Making the new node  
    struct Node* newNode = malloc(sizeof(struct Node));  
    printf("Enter the value of the new node\n");  
    scanf("%d",&newNode->value);  
    newNode->nextNode=NULL;  
  
    if(head==NULL) { // if the list is empty  
        head=newNode;  
    }  
    else { // if there are one or more nodes in the list  
        newNode->nextNode=head;  
        head=newNode;  
    }  
}
```

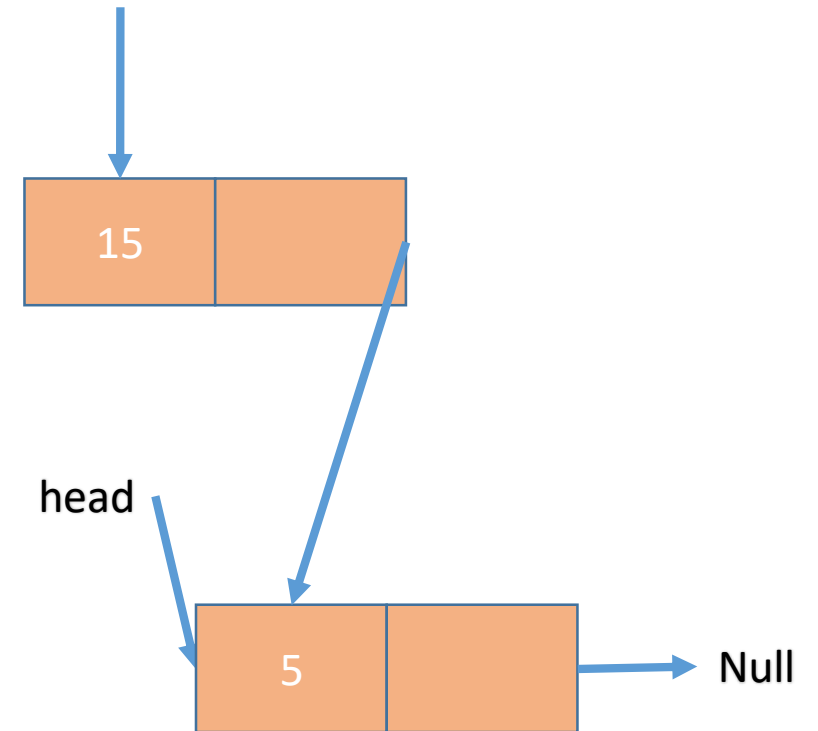
CASE 2: LIST IS NO LONGER EMPTY
newNode



Linked List Insertion (At the beginning)

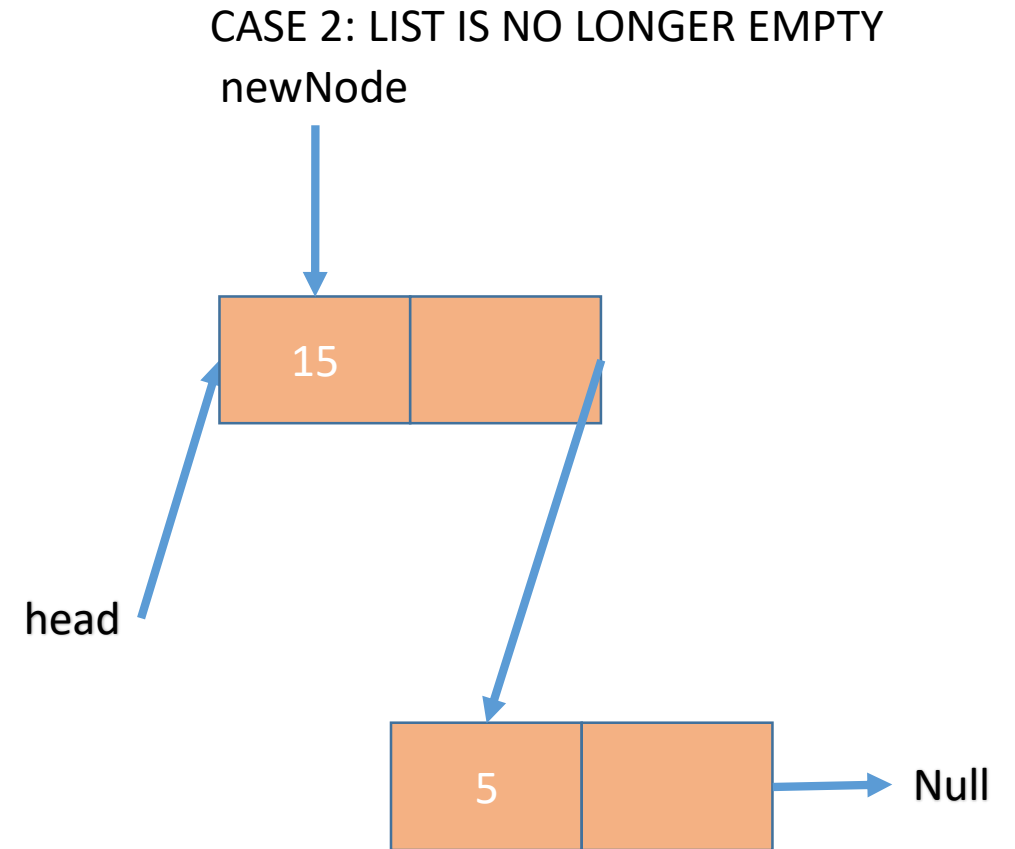
```
void addAtTheBeginning() {  
    // Making the new node  
    struct Node* newNode = malloc(sizeof(struct Node));  
    printf("Enter the value of the new node\n");  
    scanf("%d",&newNode->value);  
    newNode->nextNode=NULL;  
  
    if(head==NULL) { // if the list is empty  
        head=newNode;  
    }  
    else { // if there are one or more nodes in the list  
        → newNode->nextNode=head;  
        head=newNode;  
    }  
}
```

CASE 2: LIST IS NO LONGER EMPTY
newNode



Linked List Insertion (At the beginning)

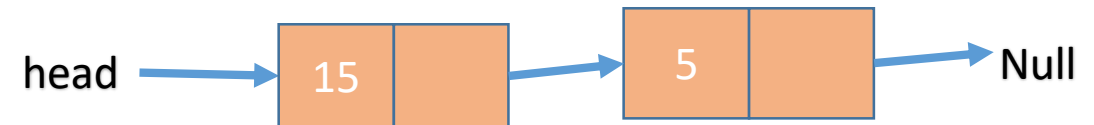
```
void addAtTheBeginning() {  
    // Making the new node  
    struct Node* newNode = malloc(sizeof(struct Node));  
    printf("Enter the value of the new node\n");  
    scanf("%d",&newNode->value);  
    newNode->nextNode=NULL;  
  
    if(head==NULL) { // if the list is empty  
        head=newNode;  
    } else { // if there are one or more nodes in the list  
        newNode->nextNode=head;  
        head=newNode;  
    }  
}
```



Linked List Insertion (At the beginning)

CASE 2: LIST IS NO LONGER EMPTY

```
void addAtTheBeginning() {  
    // Making the new node  
    struct Node* newNode = malloc(sizeof(struct Node));  
    printf("Enter the value of the new node\n");  
    scanf("%d", &newNode->value);  
    newNode->nextNode=NULL;  
  
    if(head==NULL) { // if the list is empty  
        head=newNode;  
    }  
    else { // if there are one or more nodes in the list  
        newNode->nextNode=head;  
        head=newNode;  
    }  
}
```



Linked List Insertion (At the End)

```
void addAtTheEnd() {  
    // Making the new node  
    struct Node* newNode = malloc(sizeof(struct Node));  
    printf("Enter the value of the new node\n");  
    scanf("%d", &newNode->value);  
    newNode->nextNode=NULL;  
  
    if(head==NULL) { // if the list is empty  
        head=newNode;  
    }  
    else {  
        struct Node* traveller=head;  
        while(traveller->nextNode!=NULL) {  
            traveller=traveller->nextNode;  
        }  
        traveller->nextNode=newNode;  
    }  
}
```

CASE 1: THE LIST IS EMPTY



Linked List Insertion (At the End)

```
void addAtTheEnd() {  
    // Making the new node  
    struct Node* newNode = malloc(sizeof(struct Node));  
    printf("Enter the value of the new node\n");  
    scanf("%d", &newNode->value);  
    newNode->nextNode=NULL;  
  
    if(head==NULL) { // if the list is empty  
        head=newNode;  
    }  
    else {  
        struct Node* traveller=head;  
        while(traveller->nextNode!=NULL) {  
            traveller=traveller->nextNode;  
        }  
        traveller->nextNode=newNode;  
    }  
}
```

CASE 1: THE LIST IS EMPTY
(exactly like before)

Linked List Insertion (At the End)

CASE 2: THE LIST IS NOT EMPTY

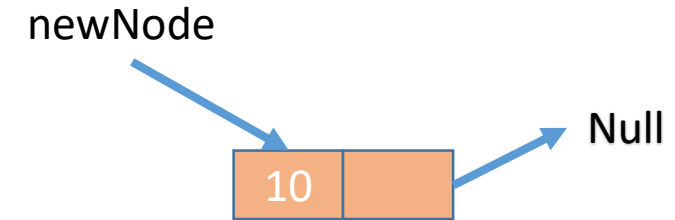
```
void addAtTheEnd() {  
    // Making the new node  
    struct Node* newNode = malloc(sizeof(struct Node));  
    printf("Enter the value of the new node\n");  
    scanf("%d", &newNode->value);  
    newNode->nextNode=NULL;  
  
    if(head==NULL) { // if the list is empty  
        head=newNode;  
    }  
    else {  
        struct Node* traveller=head;  
        while(traveller->nextNode!=NULL) {  
            traveller=traveller->nextNode;  
        }  
        traveller->nextNode=newNode;  
    }  
}
```



Linked List Insertion (At the End)

CASE 2: THE LIST IS NOT EMPTY

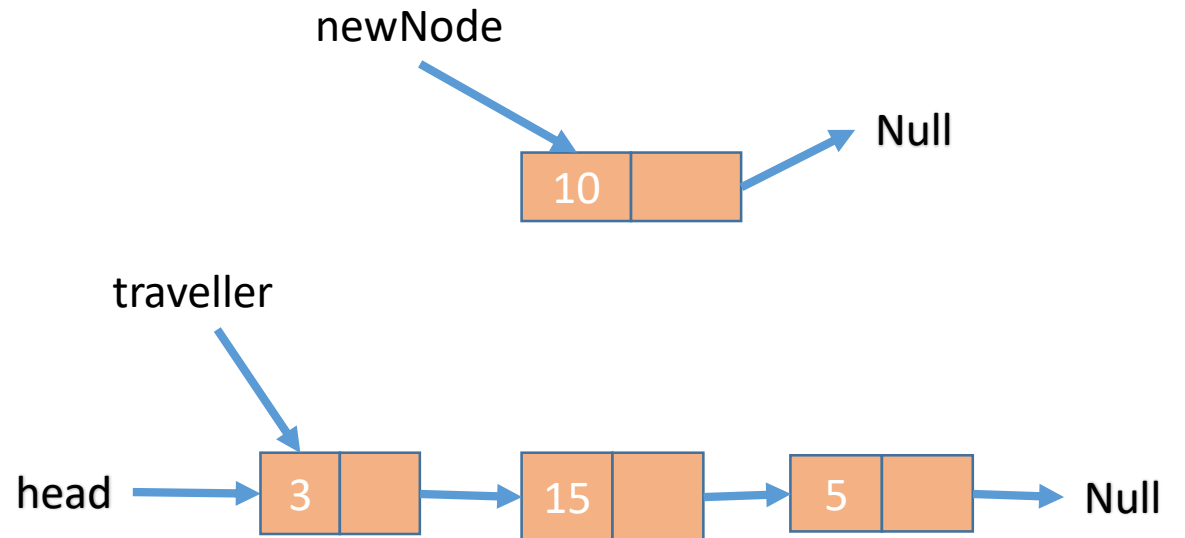
```
void addAtTheEnd() {  
    // Making the new node  
    struct Node* newNode = malloc(sizeof(struct Node));  
    printf("Enter the value of the new node\n");  
    scanf("%d", &newNode->value);  
    newNode->nextNode=NULL;  
  
    if(head==NULL) { // if the list is empty  
        head=newNode;  
    }  
    else {  
        struct Node* traveller=head;  
        while(traveller->nextNode!=NULL) {  
            traveller=traveller->nextNode;  
        }  
        traveller->nextNode=newNode;  
    }  
}
```



Linked List Insertion (At the End)

CASE 2: THE LIST IS NOT EMPTY

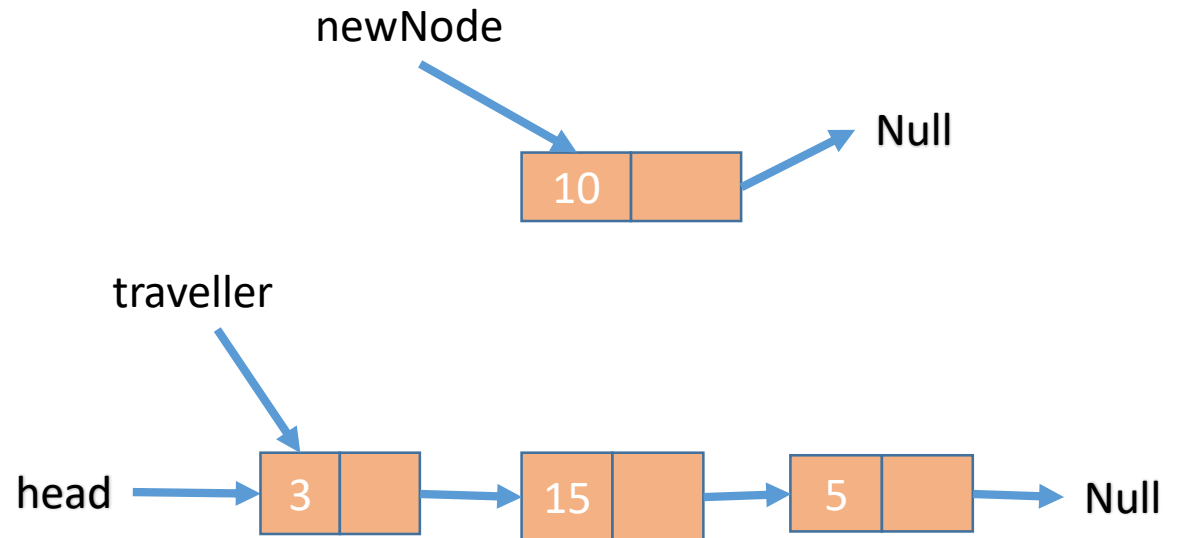
```
void addAtTheEnd() {  
    // Making the new node  
    struct Node* newNode = malloc(sizeof(struct Node));  
    printf("Enter the value of the new node\n");  
    scanf("%d", &newNode->value);  
    newNode->nextNode=NULL;  
  
    if(head==NULL) { // if the list is empty  
        head=newNode;  
    }  
    else {  
        struct Node* traveller=head;  
        while(traveller->nextNode!=NULL) {  
            traveller=traveller->nextNode;  
        }  
        traveller->nextNode=newNode;  
    }  
}
```



Linked List Insertion (At the End)

CASE 2: THE LIST IS NOT EMPTY

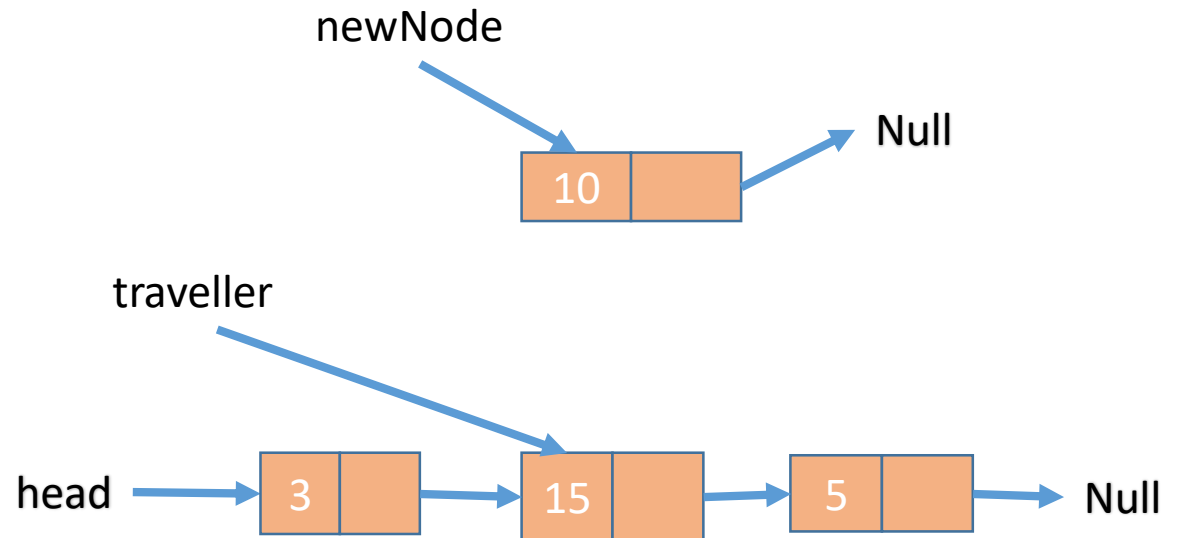
```
void addAtTheEnd() {  
    // Making the new node  
    struct Node* newNode = malloc(sizeof(struct Node));  
    printf("Enter the value of the new node\n");  
    scanf("%d", &newNode->value);  
    newNode->nextNode=NULL;  
  
    if(head==NULL) { // if the list is empty  
        head=newNode;  
    }  
    else {  
        struct Node* traveller=head;  
        while(traveller->nextNode!=NULL) {  
            traveller=traveller->nextNode;  
        }  
        traveller->nextNode=newNode;  
    }  
}
```



Linked List Insertion (At the End)

CASE 2: THE LIST IS NOT EMPTY

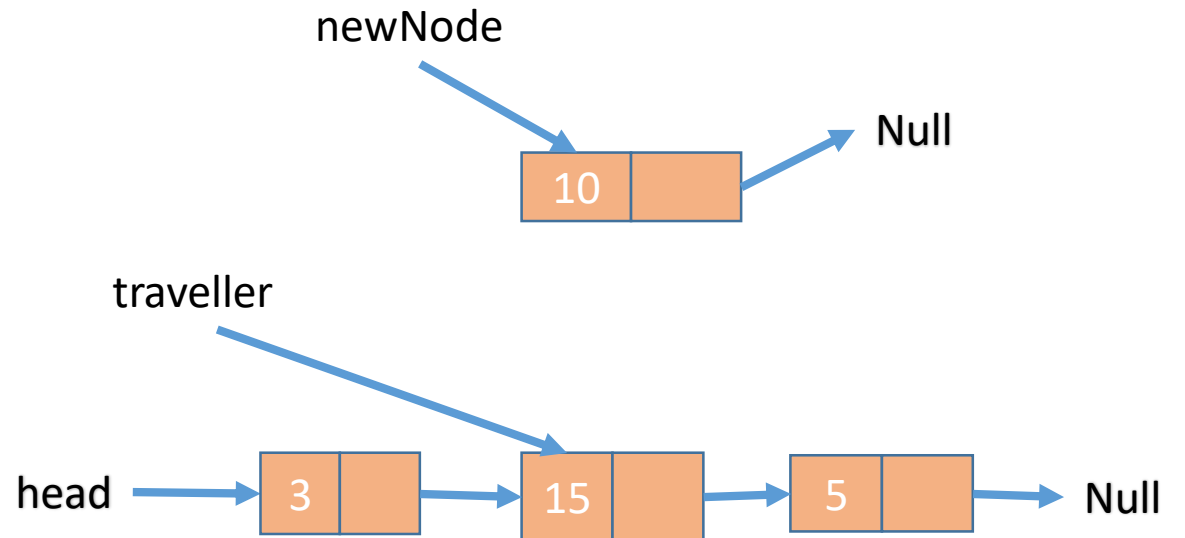
```
void addAtTheEnd() {  
    // Making the new node  
    struct Node* newNode = malloc(sizeof(struct Node));  
    printf("Enter the value of the new node\n");  
    scanf("%d", &newNode->value);  
    newNode->nextNode=NULL;  
  
    if(head==NULL) { // if the list is empty  
        head=newNode;  
    }  
    else {  
        struct Node* traveller=head;  
        while(traveller->nextNode!=NULL) {  
            traveller=traveller->nextNode;  
        }  
        traveller->nextNode=newNode;  
    }  
}
```



Linked List Insertion (At the End)

CASE 2: THE LIST IS NOT EMPTY

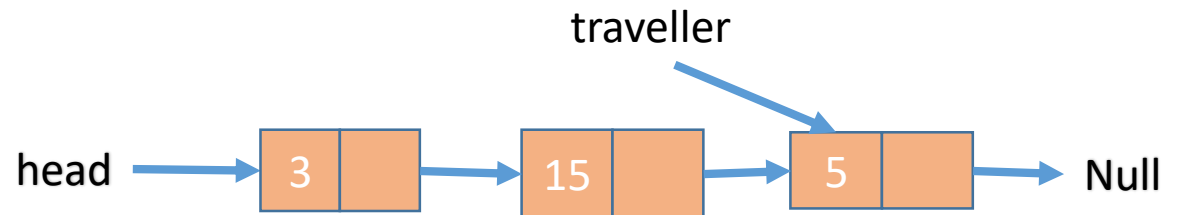
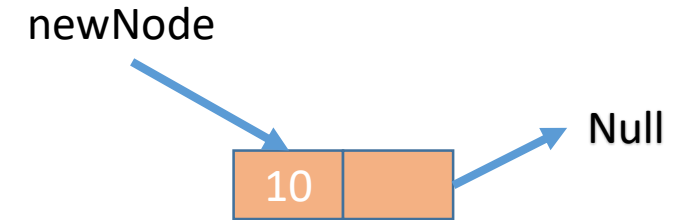
```
void addAtTheEnd() {  
    // Making the new node  
    struct Node* newNode = malloc(sizeof(struct Node));  
    printf("Enter the value of the new node\n");  
    scanf("%d", &newNode->value);  
    newNode->nextNode=NULL;  
  
    if(head==NULL) { // if the list is empty  
        head=newNode;  
    }  
    else {  
        struct Node* traveller=head;  
        while(traveller->nextNode!=NULL) {  
            traveller=traveller->nextNode;  
        }  
        traveller->nextNode=newNode;  
    }  
}
```



Linked List Insertion (At the End)

CASE 2: THE LIST IS NOT EMPTY

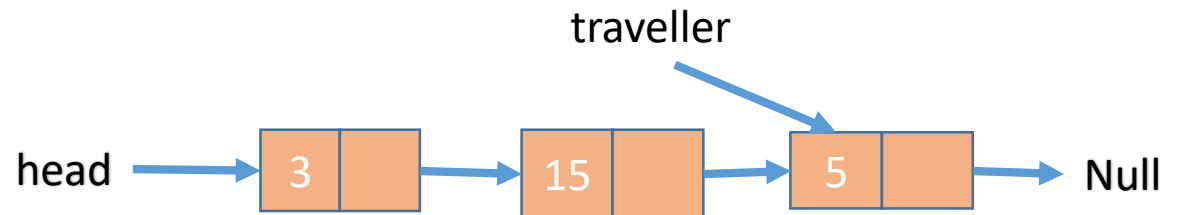
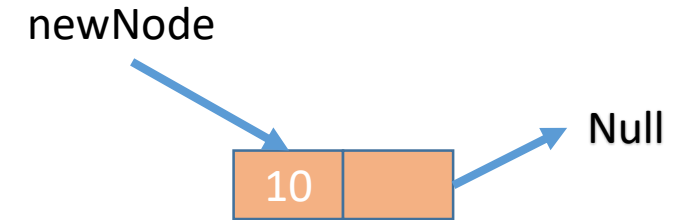
```
void addAtTheEnd() {  
    // Making the new node  
    struct Node* newNode = malloc(sizeof(struct Node));  
    printf("Enter the value of the new node\n");  
    scanf("%d", &newNode->value);  
    newNode->nextNode=NULL;  
  
    if(head==NULL) { // if the list is empty  
        head=newNode;  
    }  
    else {  
        struct Node* traveller=head;  
        while(traveller->nextNode!=NULL) {  
            traveller=traveller->nextNode;  
        }  
        traveller->nextNode=newNode;  
    }  
}
```



Linked List Insertion (At the End)

CASE 2: THE LIST IS NOT EMPTY

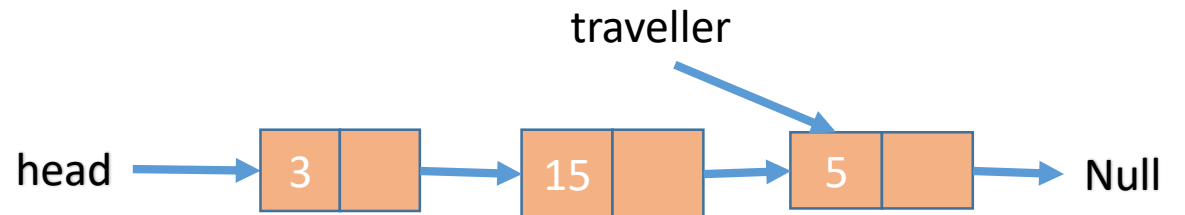
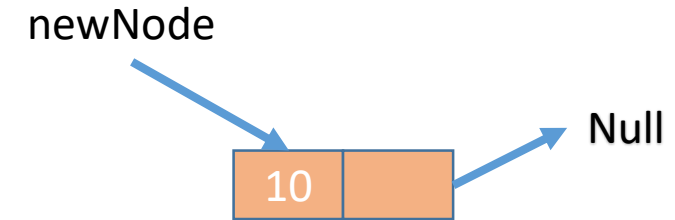
```
void addAtTheEnd() {  
    // Making the new node  
    struct Node* newNode = malloc(sizeof(struct Node));  
    printf("Enter the value of the new node\n");  
    scanf("%d", &newNode->value);  
    newNode->nextNode=NULL;  
  
    if(head==NULL) { // if the list is empty  
        head=newNode;  
    }  
    else {  
        struct Node* traveller=head;  
        while(traveller->nextNode!=NULL) {  
            traveller=traveller->nextNode;  
        }  
        traveller->nextNode=newNode;  
    }  
}
```



Linked List Insertion (At the End)

CASE 2: THE LIST IS NOT EMPTY

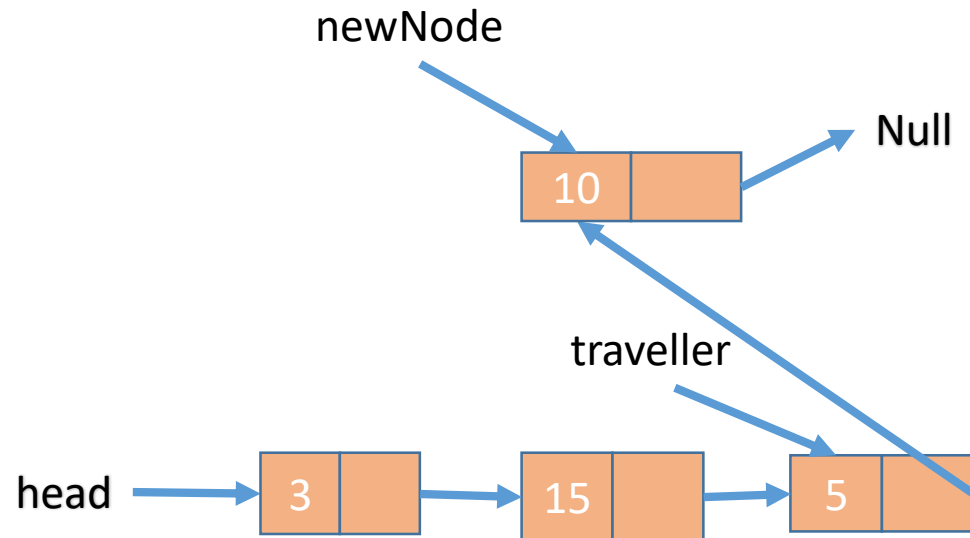
```
void addAtTheEnd() {  
    // Making the new node  
    struct Node* newNode = malloc(sizeof(struct Node));  
    printf("Enter the value of the new node\n");  
    scanf("%d", &newNode->value);  
    newNode->nextNode=NULL;  
  
    if(head==NULL) { // if the list is empty  
        head=newNode;  
    }  
    else {  
        struct Node* traveller=head;  
        while(traveller->nextNode!=NULL) {  
            traveller=traveller->nextNode;  
        }  
        traveller->nextNode=newNode;  
    }  
}
```



Linked List Insertion (At the End)

CASE 2: THE LIST IS NOT EMPTY

```
void addAtTheEnd() {  
    // Making the new node  
    struct Node* newNode = malloc(sizeof(struct Node));  
    printf("Enter the value of the new node\n");  
    scanf("%d", &newNode->value);  
    newNode->nextNode=NULL;  
  
    if(head==NULL) { // if the list is empty  
        head=newNode;  
    }  
    else {  
        struct Node* traveller=head;  
        while(traveller->nextNode!=NULL) {  
            traveller=traveller->nextNode;  
        }  
        traveller->nextNode=newNode;  
    }  
}
```



Linked List Insertion (At the End)

CASE 2: THE LIST IS NOT EMPTY

```
void addAtTheEnd() {  
    // Making the new node  
    struct Node* newNode = malloc(sizeof(struct Node));  
    printf("Enter the value of the new node\n");  
    scanf("%d", &newNode->value);  
    newNode->nextNode=NULL;  
  
    if(head==NULL) { // if the list is empty  
        head=newNode;  
    }  
    else {  
        struct Node* traveller=head;  
        while(traveller->nextNode!=NULL) {  
            traveller=traveller->nextNode;  
        }  
        traveller->nextNode=newNode;  
    }  
}
```



Linked List Insertion (After a particular Node)

Linked List Insertion (After a particular Node)

```
void addAfterAnElement() {
    if (head==NULL) {
        printf("The list is empty!");
        return;
    }
    // Making the new node by taking the node value from the user
    struct Node* newNode = malloc(sizeof(struct Node));
    printf("Enter the value of the new node\n");
    scanf("%d",&newNode->value);
    newNode->nextNode=NULL;
    // Taking the value of the node after which we will insert
    printf("Enter the value of the node after which you want to enter the new node\n");
    int valueOfThePredecessorNode;
    scanf("%d",&valueOfThePredecessorNode);
    // finding that particular node
    struct Node* traveller;
    traveller=head;
    while(traveller!=NULL) {
        if(traveller->value==valueOfThePredecessorNode) {
            newNode->nextNode=traveller->nextNode;
            traveller->nextNode=newNode;
            return;
        }
        traveller=traveller->nextNode;
    }
    printf("The node after which you wanted to insert does not exist!\n");
}
```


Linked List Insertion (After a particular Node)

```
void addAfterAnElement() {
    if (head==NULL) {
        printf("The list is empty!");
        return;
    }
    // Making the new node by taking the node value from the user
    struct Node* newNode = malloc(sizeof(struct Node));
    printf("Enter the value of the new node\n");
    scanf("%d",&newNode->value);
    newNode->nextNode=NULL;
    // Taking the value of the node after which we will insert
    printf("Enter the value of the node after which you want to enter the new node\n");
    int valueOfThePredecessorNode;
    scanf("%d",&valueOfThePredecessorNode);
    // finding that particular node
    struct Node* traveller;
    traveller=head;
    while(traveller!=NULL) {
        if(traveller->value==valueOfThePredecessorNode) {
            newNode->nextNode=traveller->nextNode;
            traveller->nextNode=newNode;
            return;
        }
        traveller=traveller->nextNode;
    }
    printf("The node after which you wanted to insert does not exist!\n");
}
```

CASE 1: LIST IS EMPTY



Linked List Insertion (After a particular Node)

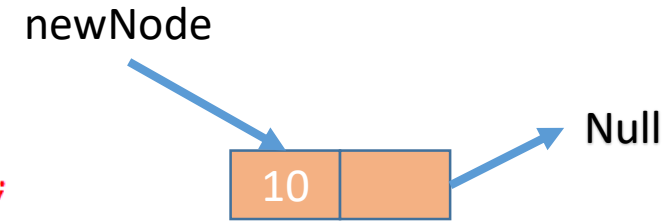
```
void addAfterAnElement() {
    if (head==NULL) {
        printf("The list is empty!");
        return;
    }
    // Making the new node by taking the node value from the user
    struct Node* newNode = malloc(sizeof(struct Node));
    printf("Enter the value of the new node\n");
    scanf("%d",&newNode->value);
    newNode->nextNode=NULL;
    // Taking the value of the node after which we will insert
    printf("Enter the value of the node after which you want to enter the new node\n");
    int valueOfThePredecessorNode;
    scanf("%d",&valueOfThePredecessorNode);
    // finding that particular node
    struct Node* traveller;
    traveller=head;
    while(traveller!=NULL) {
        if(traveller->value==valueOfThePredecessorNode) {
            newNode->nextNode=traveller->nextNode;
            traveller->nextNode=newNode;
            return;
        }
        traveller=traveller->nextNode;
    }
    printf("The node after which you wanted to insert does not exist!\n");
}
```

CASE 2: LIST IS NOT EMPTY & ELEMENT EXISTS

Linked List Insertion (After a particular Node)

```
void addAfterAnElement() {
    if (head==NULL) {
        printf("The list is empty!");
        return;
    }
    // Making the new node by taking the node value from the user
    struct Node* newNode = malloc(sizeof(struct Node));
    printf("Enter the value of the new node\n");
    scanf("%d",&newNode->value);
    newNode->nextNode=NULL;
    // Taking the value of the node after which we will insert
    printf("Enter the value of the node after which you want to enter the new node\n");
    int valueOfThePredecessorNode;
    scanf("%d",&valueOfThePredecessorNode);
    // finding that particular node
    struct Node* traveller;
    traveller=head;
    while(traveller!=NULL) {
        if(traveller->value==valueOfThePredecessorNode) {
            newNode->nextNode=traveller->nextNode;
            traveller->nextNode=newNode;
            return;
        }
        traveller=traveller->nextNode;
    }
    printf("The node after which you wanted to insert does not exist!\n");
}
```

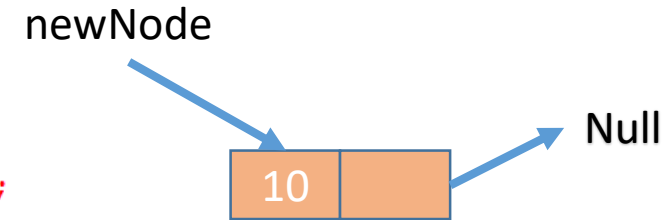
CASE 2: LIST IS NOT EMPTY & ELEMENT EXISTS



Linked List Insertion (After a particular Node)

```
void addAfterAnElement() {
    if (head==NULL) {
        printf("The list is empty!");
        return;
    }
    // Making the new node by taking the node value from the user
    struct Node* newNode = malloc(sizeof(struct Node));
    printf("Enter the value of the new node\n");
    scanf("%d",&newNode->value);
    newNode->nextNode=NULL;
    // Taking the value of the node after which we will insert
    printf("Enter the value of the node after which you want to enter the new node\n");
    int valueOfThePredecessorNode;
    scanf("%d",&valueOfThePredecessorNode);
    // finding that particular node
    struct Node* traveller;
    traveller=head;
    while(traveller!=NULL) {
        if(traveller->value==valueOfThePredecessorNode) {
            newNode->nextNode=traveller->nextNode;
            traveller->nextNode=newNode;
            return;
        }
        traveller=traveller->nextNode;
    }
    printf("The node after which you wanted to insert does not exist!\n");
}
```

CASE 2: LIST IS NOT EMPTY & ELEMENT EXISTS



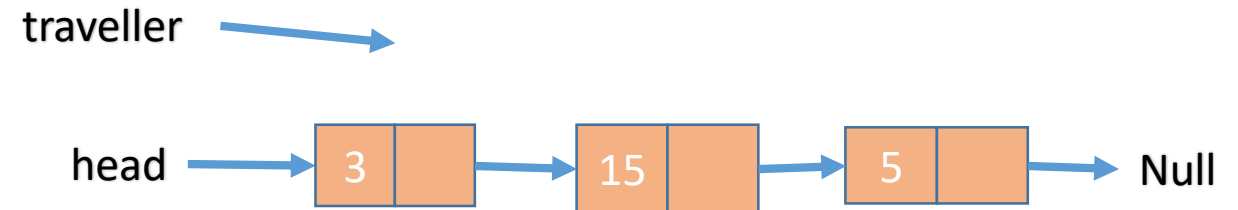
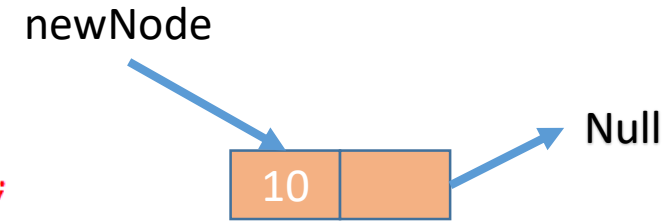
Assume that we want to insert after 15



Linked List Insertion (After a particular Node)

```
void addAfterAnElement() {
    if (head==NULL) {
        printf("The list is empty!");
        return;
    }
    // Making the new node by taking the node value from the user
    struct Node* newNode = malloc(sizeof(struct Node));
    printf("Enter the value of the new node\n");
    scanf("%d",&newNode->value);
    newNode->nextNode=NULL;
    // Taking the value of the node after which we will insert
    printf("Enter the value of the node after which you want to enter the new node\n");
    int valueOfThePredecessorNode;
    scanf("%d",&valueOfThePredecessorNode);
    // finding that particular node
    struct Node* traveller;
    traveller=head;
    while(traveller!=NULL) {
        if(traveller->value==valueOfThePredecessorNode) {
            newNode->nextNode=traveller->nextNode;
            traveller->nextNode=newNode;
            return;
        }
        traveller=traveller->nextNode;
    }
    printf("The node after which you wanted to insert does not exist!\n");
}
```

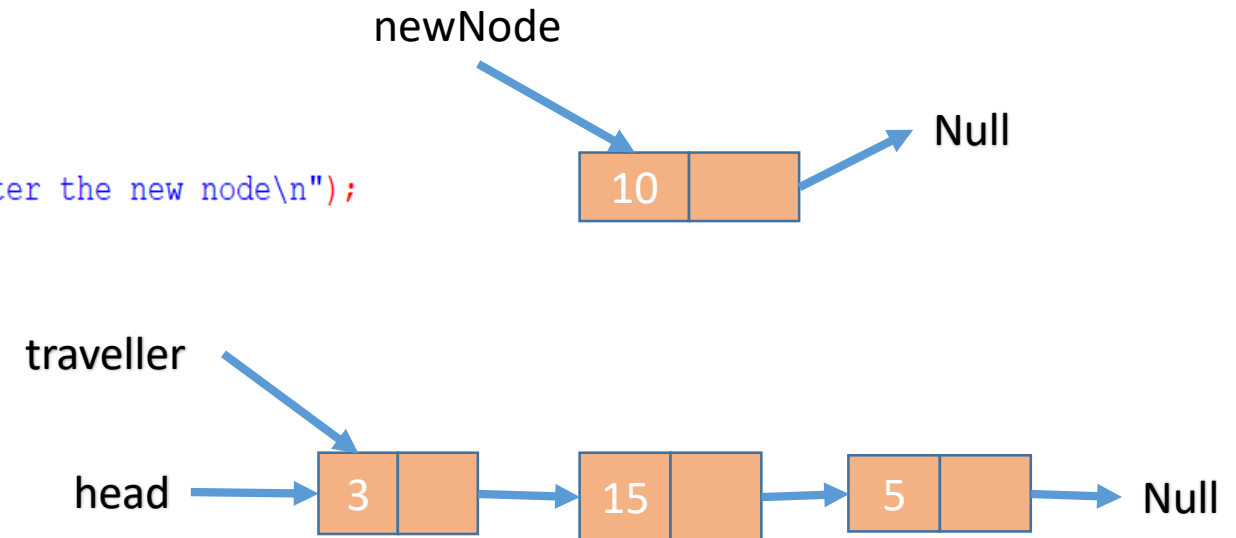
CASE 2: LIST IS NOT EMPTY & ELEMENT EXISTS



Linked List Insertion (After a particular Node)

```
void addAfterAnElement() {
    if (head==NULL) {
        printf("The list is empty!");
        return;
    }
    // Making the new node by taking the node value from the user
    struct Node* newNode = malloc(sizeof(struct Node));
    printf("Enter the value of the new node\n");
    scanf("%d",&newNode->value);
    newNode->nextNode=NULL;
    // Taking the value of the node after which we will insert
    printf("Enter the value of the node after which you want to enter the new node\n");
    int valueOfThePredecessorNode;
    scanf("%d",&valueOfThePredecessorNode);
    // finding that particular node
    struct Node* traveller;
    →traveller=head;
    while(traveller!=NULL) {
        if(traveller->value==valueOfThePredecessorNode) {
            newNode->nextNode=traveller->nextNode;
            traveller->nextNode=newNode;
            return;
        }
        traveller=traveller->nextNode;
    }
    printf("The node after which you wanted to insert does not exist!\n");
}
```

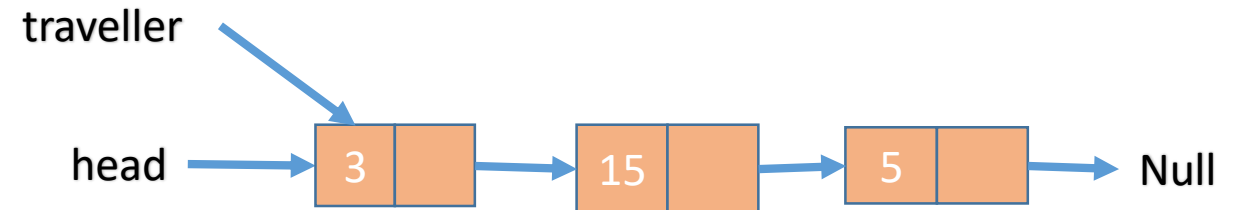
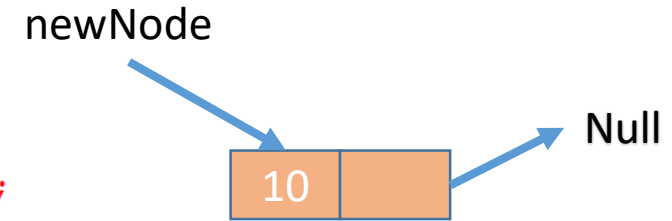
CASE 2: LIST IS NOT EMPTY & ELEMENT EXISTS



Linked List Insertion (After a particular Node)

```
void addAfterAnElement() {
    if (head==NULL) {
        printf("The list is empty!");
        return;
    }
    // Making the new node by taking the node value from the user
    struct Node* newNode = malloc(sizeof(struct Node));
    printf("Enter the value of the new node\n");
    scanf("%d",&newNode->value);
    newNode->nextNode=NULL;
    // Taking the value of the node after which we will insert
    printf("Enter the value of the node after which you want to enter the new node\n");
    int valueOfThePredecessorNode;
    scanf("%d",&valueOfThePredecessorNode);
    // finding that particular node
    struct Node* traveller;
    traveller=head;
    while(traveller!=NULL) {
        if(traveller->value==valueOfThePredecessorNode) {
            newNode->nextNode=traveller->nextNode;
            traveller->nextNode=newNode;
            return;
        }
        traveller=traveller->nextNode;
    }
    printf("The node after which you wanted to insert does not exist!\n");
}
```

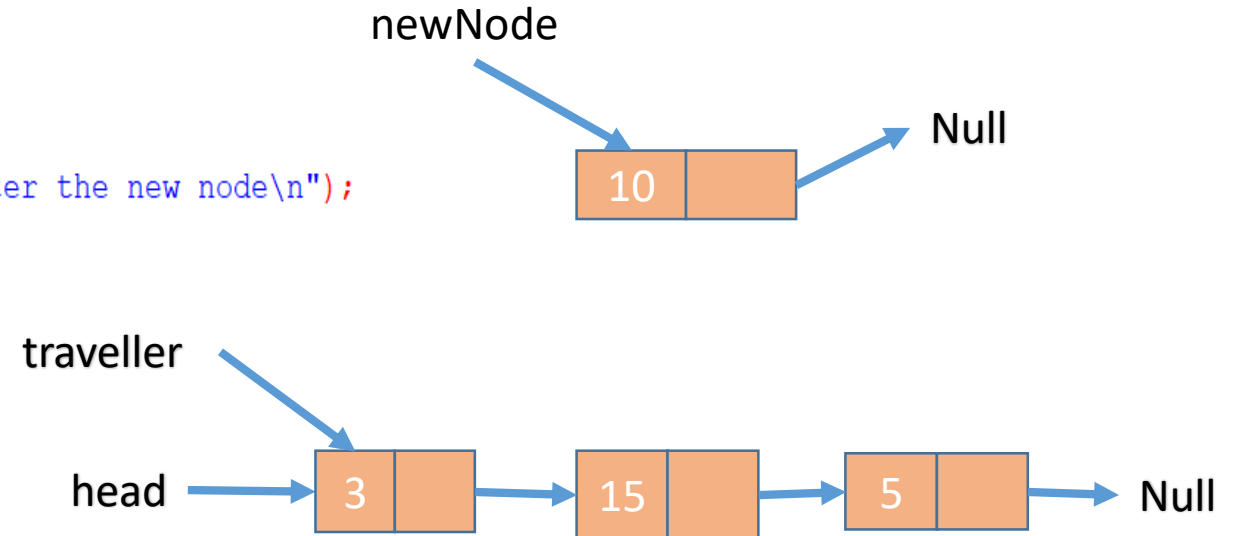
CASE 2: LIST IS NOT EMPTY & ELEMENT EXISTS



Linked List Insertion (After a particular Node)

```
void addAfterAnElement() {
    if (head==NULL) {
        printf("The list is empty!");
        return;
    }
    // Making the new node by taking the node value from the user
    struct Node* newNode = malloc(sizeof(struct Node));
    printf("Enter the value of the new node\n");
    scanf("%d",&newNode->value);
    newNode->nextNode=NULL;
    // Taking the value of the node after which we will insert
    printf("Enter the value of the node after which you want to enter the new node\n");
    int valueOfThePredecessorNode;
    scanf("%d",&valueOfThePredecessorNode);
    // finding that particular node
    struct Node* traveller;
    traveller=head;
    while(traveller!=NULL) {
        → if(traveller->value==valueOfThePredecessorNode) {
            newNode->nextNode=traveller->nextNode;
            traveller->nextNode=newNode;
            return;
        }
        traveller=traveller->nextNode;
    }
    printf("The node after which you wanted to insert does not exist!\n");
}
```

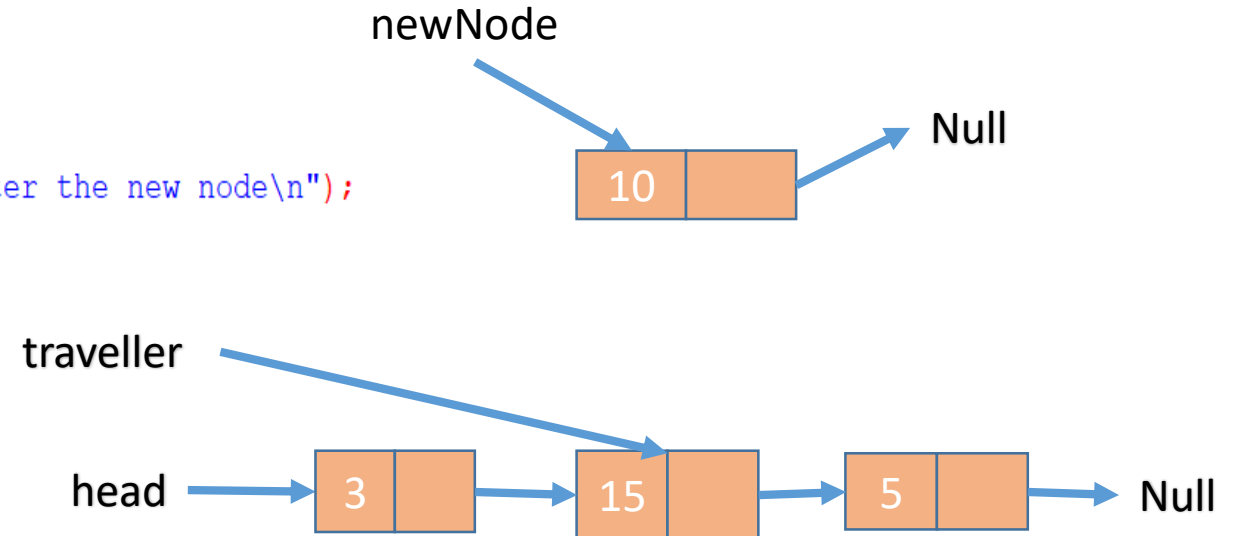
CASE 2: LIST IS NOT EMPTY & ELEMENT EXISTS



Linked List Insertion (After a particular Node)

```
void addAfterAnElement() {
    if (head==NULL) {
        printf("The list is empty!");
        return;
    }
    // Making the new node by taking the node value from the user
    struct Node* newNode = malloc(sizeof(struct Node));
    printf("Enter the value of the new node\n");
    scanf("%d",&newNode->value);
    newNode->nextNode=NULL;
    // Taking the value of the node after which we will insert
    printf("Enter the value of the node after which you want to enter the new node\n");
    int valueOfThePredecessorNode;
    scanf("%d",&valueOfThePredecessorNode);
    // finding that particular node
    struct Node* traveller;
    traveller=head;
    while(traveller!=NULL) {
        if(traveller->value==valueOfThePredecessorNode) {
            newNode->nextNode=traveller->nextNode;
            traveller->nextNode=newNode;
            return;
        }
        traveller=traveller->nextNode;
    }
    printf("The node after which you wanted to insert does not exist!\n");
}
```

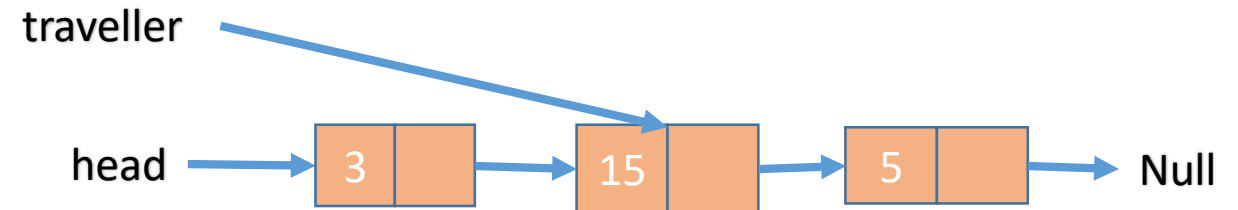
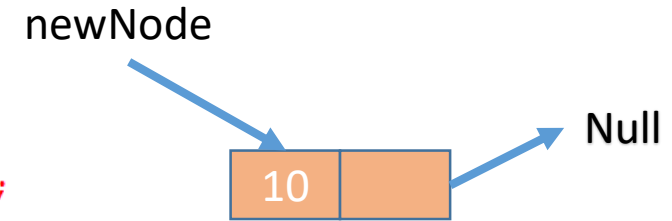
CASE 2: LIST IS NOT EMPTY & ELEMENT EXISTS



Linked List Insertion (After a particular Node)

```
void addAfterAnElement() {
    if (head==NULL) {
        printf("The list is empty!");
        return;
    }
    // Making the new node by taking the node value from the user
    struct Node* newNode = malloc(sizeof(struct Node));
    printf("Enter the value of the new node\n");
    scanf("%d",&newNode->value);
    newNode->nextNode=NULL;
    // Taking the value of the node after which we will insert
    printf("Enter the value of the node after which you want to enter the new node\n");
    int valueOfThePredecessorNode;
    scanf("%d",&valueOfThePredecessorNode);
    // finding that particular node
    struct Node* traveller;
    traveller=head;
    while(traveller!=NULL) {
        if(traveller->value==valueOfThePredecessorNode) {
            newNode->nextNode=traveller->nextNode;
            traveller->nextNode=newNode;
            return;
        }
        traveller=traveller->nextNode;
    }
    printf("The node after which you wanted to insert does not exist!\n");
}
```

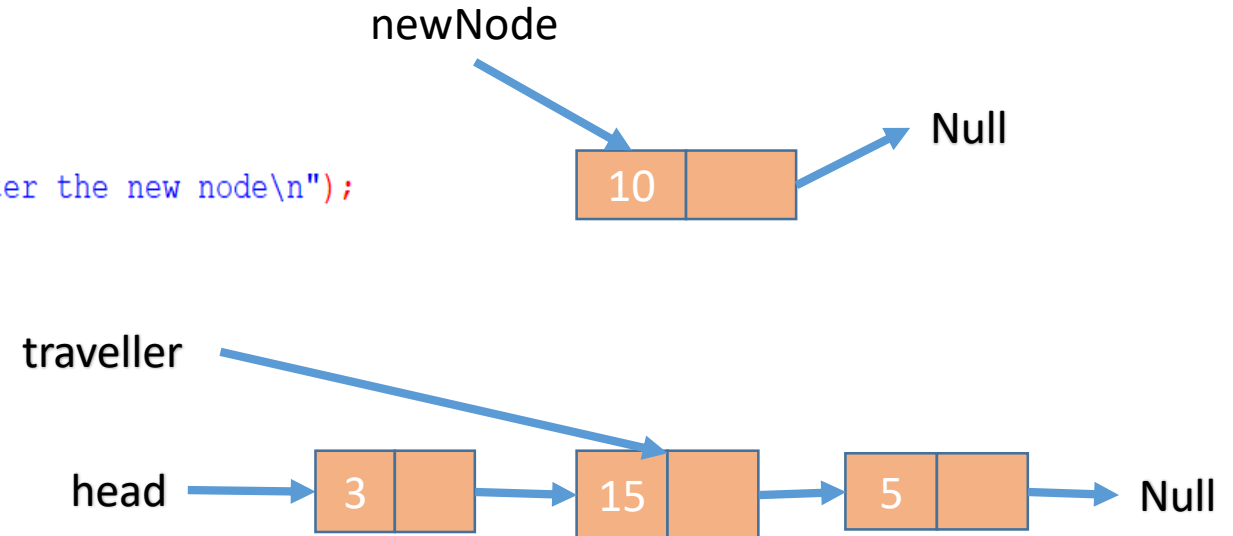
CASE 2: LIST IS NOT EMPTY & ELEMENT EXISTS



Linked List Insertion (After a particular Node)

```
void addAfterAnElement() {
    if (head==NULL) {
        printf("The list is empty!");
        return;
    }
    // Making the new node by taking the node value from the user
    struct Node* newNode = malloc(sizeof(struct Node));
    printf("Enter the value of the new node\n");
    scanf("%d",&newNode->value);
    newNode->nextNode=NULL;
    // Taking the value of the node after which we will insert
    printf("Enter the value of the node after which you want to enter the new node\n");
    int valueOfThePredecessorNode;
    scanf("%d",&valueOfThePredecessorNode);
    // finding that particular node
    struct Node* traveller;
    traveller=head;
    while(traveller!=NULL) {
        → if(traveller->value==valueOfThePredecessorNode) {
            newNode->nextNode=traveller->nextNode;
            traveller->nextNode=newNode;
            return;
        }
        traveller=traveller->nextNode;
    }
    printf("The node after which you wanted to insert does not exist!\n");
}
```

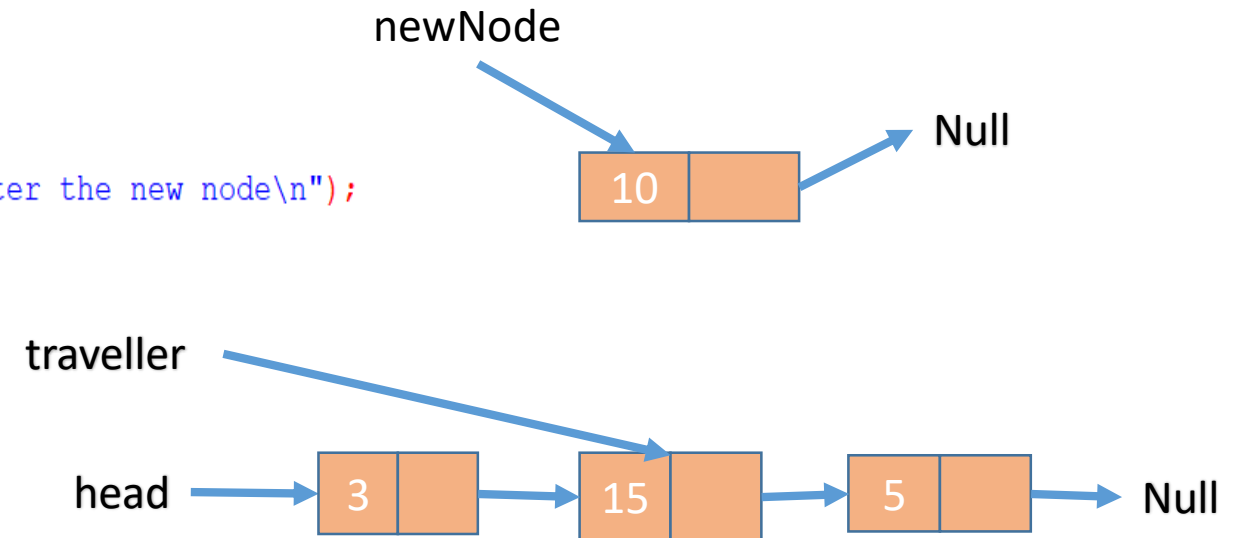
CASE 2: LIST IS NOT EMPTY & ELEMENT EXISTS



Linked List Insertion (After a particular Node)

```
void addAfterAnElement() {
    if (head==NULL) {
        printf("The list is empty!");
        return;
    }
    // Making the new node by taking the node value from the user
    struct Node* newNode = malloc(sizeof(struct Node));
    printf("Enter the value of the new node\n");
    scanf("%d",&newNode->value);
    newNode->nextNode=NULL;
    // Taking the value of the node after which we will insert
    printf("Enter the value of the node after which you want to enter the new node\n");
    int valueOfThePredecessorNode;
    scanf("%d",&valueOfThePredecessorNode);
    // finding that particular node
    struct Node* traveller;
    traveller=head;
    while(traveller!=NULL) {
        if(traveller->value==valueOfThePredecessorNode) {
            newNode->nextNode=traveller->nextNode;
            traveller->nextNode=newNode;
            return;
        }
        traveller=traveller->nextNode;
    }
    printf("The node after which you wanted to insert does not exist!\n");
}
```

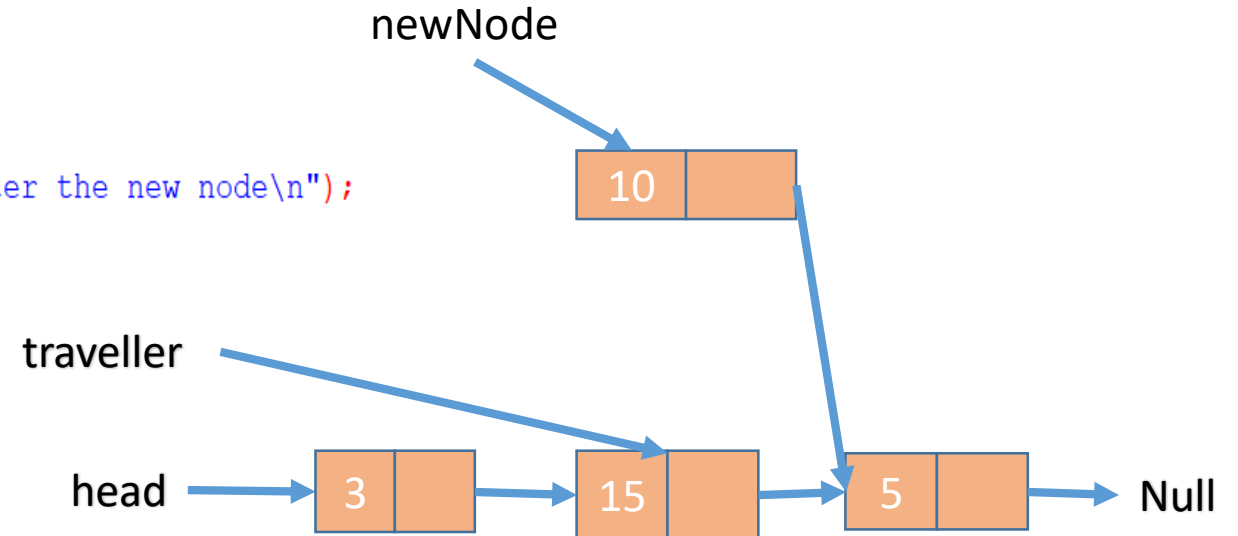
CASE 2: LIST IS NOT EMPTY & ELEMENT EXISTS



Linked List Insertion (After a particular Node)

```
void addAfterAnElement() {
    if (head==NULL) {
        printf("The list is empty!");
        return;
    }
    // Making the new node by taking the node value from the user
    struct Node* newNode = malloc(sizeof(struct Node));
    printf("Enter the value of the new node\n");
    scanf("%d",&newNode->value);
    newNode->nextNode=NULL;
    // Taking the value of the node after which we will insert
    printf("Enter the value of the node after which you want to enter the new node\n");
    int valueOfThePredecessorNode;
    scanf("%d",&valueOfThePredecessorNode);
    // finding that particular node
    struct Node* traveller;
    traveller=head;
    while(traveller!=NULL) {
        if(traveller->value==valueOfThePredecessorNode) {
            newNode->nextNode=traveller->nextNode;
            traveller->nextNode=newNode;
            return;
        }
        traveller=traveller->nextNode;
    }
    printf("The node after which you wanted to insert does not exist!\n");
}
```

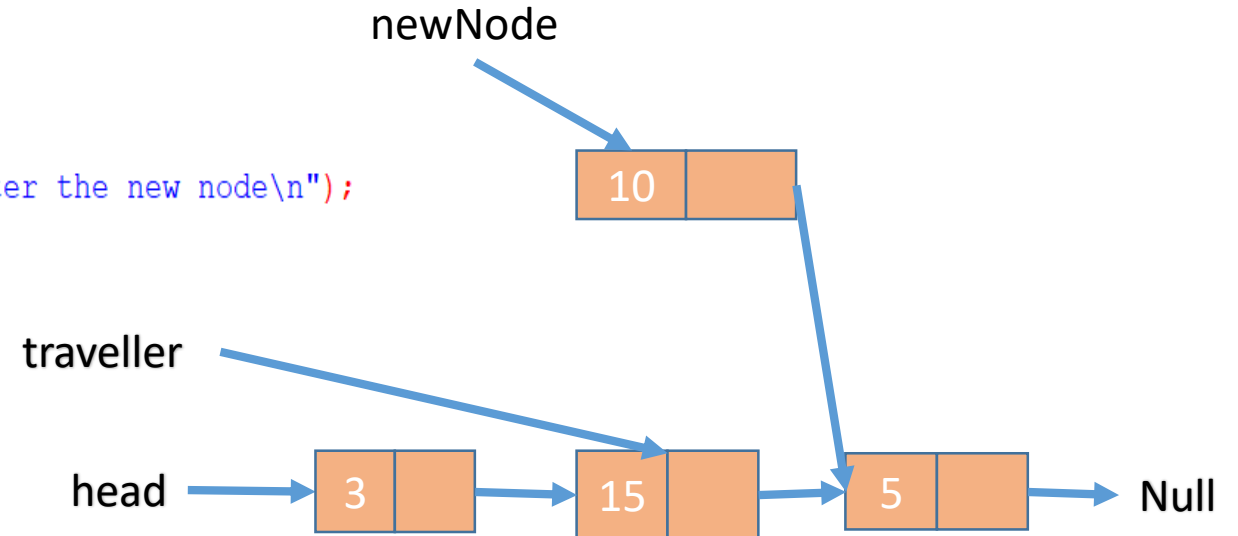
CASE 2: LIST IS NOT EMPTY & ELEMENT EXISTS



Linked List Insertion (After a particular Node)

```
void addAfterAnElement() {
    if (head==NULL) {
        printf("The list is empty!");
        return;
    }
    // Making the new node by taking the node value from the user
    struct Node* newNode = malloc(sizeof(struct Node));
    printf("Enter the value of the new node\n");
    scanf("%d",&newNode->value);
    newNode->nextNode=NULL;
    // Taking the value of the node after which we will insert
    printf("Enter the value of the node after which you want to enter the new node\n");
    int valueOfThePredecessorNode;
    scanf("%d",&valueOfThePredecessorNode);
    // finding that particular node
    struct Node* traveller;
    traveller=head;
    while(traveller!=NULL) {
        if(traveller->value==valueOfThePredecessorNode) {
            newNode->nextNode=traveller->nextNode;
            traveller->nextNode=newNode;
            return;
        }
        traveller=traveller->nextNode;
    }
    printf("The node after which you wanted to insert does not exist!\n");
}
```

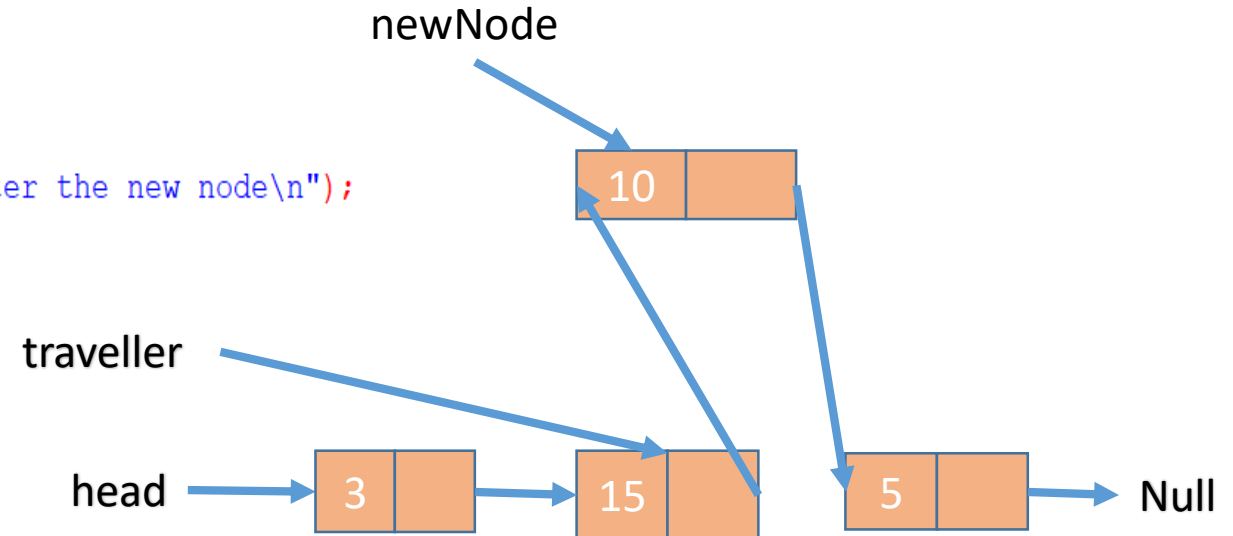
CASE 2: LIST IS NOT EMPTY & ELEMENT EXISTS



Linked List Insertion (After a particular Node)

```
void addAfterAnElement() {
    if (head==NULL) {
        printf("The list is empty!");
        return;
    }
    // Making the new node by taking the node value from the user
    struct Node* newNode = malloc(sizeof(struct Node));
    printf("Enter the value of the new node\n");
    scanf("%d",&newNode->value);
    newNode->nextNode=NULL;
    // Taking the value of the node after which we will insert
    printf("Enter the value of the node after which you want to enter the new node\n");
    int valueOfThePredecessorNode;
    scanf("%d",&valueOfThePredecessorNode);
    // finding that particular node
    struct Node* traveller;
    traveller=head;
    while(traveller!=NULL) {
        if(traveller->value==valueOfThePredecessorNode) {
            newNode->nextNode=traveller->nextNode;
            traveller->nextNode=newNode;
            return;
        }
        traveller=traveller->nextNode;
    }
    printf("The node after which you wanted to insert does not exist!\n");
}
```

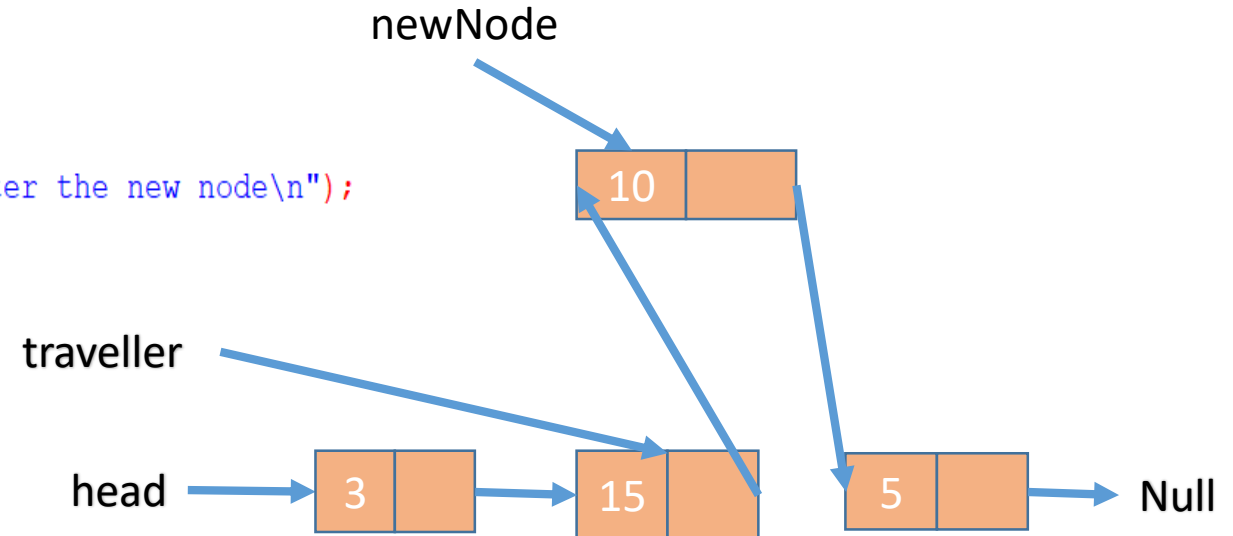
CASE 2: LIST IS NOT EMPTY & ELEMENT EXISTS



Linked List Insertion (After a particular Node)

```
void addAfterAnElement() {
    if (head==NULL) {
        printf("The list is empty!");
        return;
    }
    // Making the new node by taking the node value from the user
    struct Node* newNode = malloc(sizeof(struct Node));
    printf("Enter the value of the new node\n");
    scanf("%d",&newNode->value);
    newNode->nextNode=NULL;
    // Taking the value of the node after which we will insert
    printf("Enter the value of the node after which you want to enter the new node\n");
    int valueOfThePredecessorNode;
    scanf("%d",&valueOfThePredecessorNode);
    // finding that particular node
    struct Node* traveller;
    traveller=head;
    while(traveller!=NULL) {
        if(traveller->value==valueOfThePredecessorNode) {
            newNode->nextNode=traveller->nextNode;
            traveller->nextNode=newNode;
            return;
        }
        traveller=traveller->nextNode;
    }
    printf("The node after which you wanted to insert does not exist!\n");
}
```

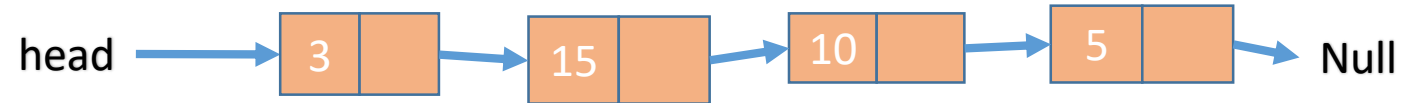
CASE 2: LIST IS NOT EMPTY & ELEMENT EXISTS



Linked List Insertion (After a particular Node)

```
void addAfterAnElement() {
    if (head==NULL) {
        printf("The list is empty!");
        return;
    }
    // Making the new node by taking the node value from the user
    struct Node* newNode = malloc(sizeof(struct Node));
    printf("Enter the value of the new node\n");
    scanf("%d",&newNode->value);
    newNode->nextNode=NULL;
    // Taking the value of the node after which we will insert
    printf("Enter the value of the node after which you want to enter the new node\n");
    int valueOfThePredecessorNode;
    scanf("%d",&valueOfThePredecessorNode);
    // finding that particular node
    struct Node* traveller;
    traveller=head;
    while(traveller!=NULL) {
        if(traveller->value==valueOfThePredecessorNode) {
            newNode->nextNode=traveller->nextNode;
            traveller->nextNode=newNode;
            return;
        }
        traveller=traveller->nextNode;
    }
    printf("The node after which you wanted to insert does not exist!\n");
}
```

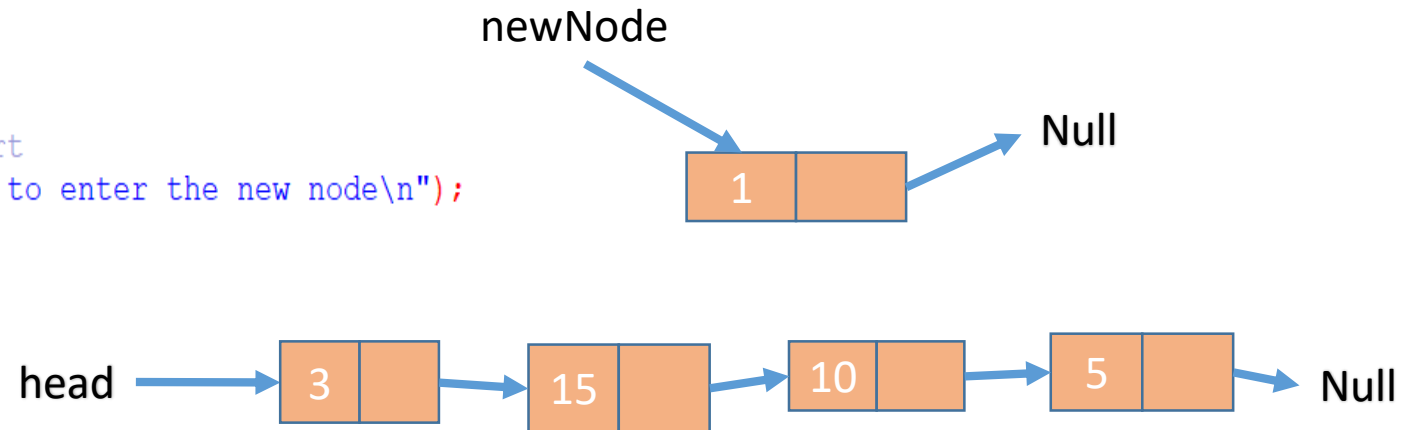
CASE 2: LIST IS NOT EMPTY & ELEMENT EXISTS



Linked List Insertion (After a particular Node)

```
void addAfterAnElement() {
    if (head==NULL) {
        printf("The list is empty!");
        return;
    }
    // Making the new node by taking the node value from the user
    struct Node* newNode = malloc(sizeof(struct Node));
    printf("Enter the value of the new node\n");
    scanf("%d",&newNode->value);
    newNode->nextNode=NULL;
    // Taking the value of the node after which we will insert
    printf("Enter the value of the node after which you want to enter the new node\n");
    int valueOfThePredecessorNode;
    scanf("%d",&valueOfThePredecessorNode);
    // finding that particular node
    struct Node* traveller;
    traveller=head;
    while(traveller!=NULL) {
        if(traveller->value==valueOfThePredecessorNode) {
            newNode->nextNode=traveller->nextNode;
            traveller->nextNode=newNode;
            return;
        }
        traveller=traveller->nextNode;
    }
    printf("The node after which you wanted to insert does not exist!\n");
}
```

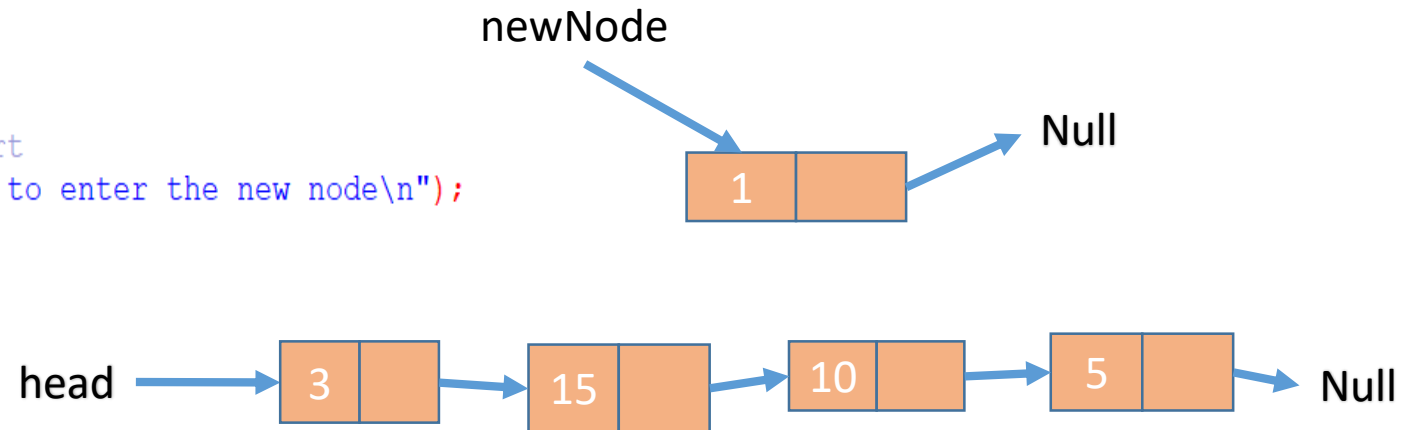
**CASE 3: LIST IS NOT EMPTY &
DOES NOT ELEMENT EXISTS**



Linked List Insertion (After a particular Node)

```
void addAfterAnElement() {
    if (head==NULL) {
        printf("The list is empty!");
        return;
    }
    // Making the new node by taking the node value from the user
    struct Node* newNode = malloc(sizeof(struct Node));
    printf("Enter the value of the new node\n");
    scanf("%d",&newNode->value);
    newNode->nextNode=NULL;
    // Taking the value of the node after which we will insert
    printf("Enter the value of the node after which you want to enter the new node\n");
    int valueOfThePredecessorNode;
    scanf("%d",&valueOfThePredecessorNode);
    // finding that particular node
    struct Node* traveller;
    traveller=head;
    while(traveller!=NULL) {
        if(traveller->value==valueOfThePredecessorNode) {
            newNode->nextNode=traveller->nextNode;
            traveller->nextNode=newNode;
            return;
        }
        traveller=traveller->nextNode;
    }
    printf("The node after which you wanted to insert does not exist!\n");
}
```

**CASE 3: LIST IS NOT EMPTY &
DOES NOT ELEMENT EXISTS**

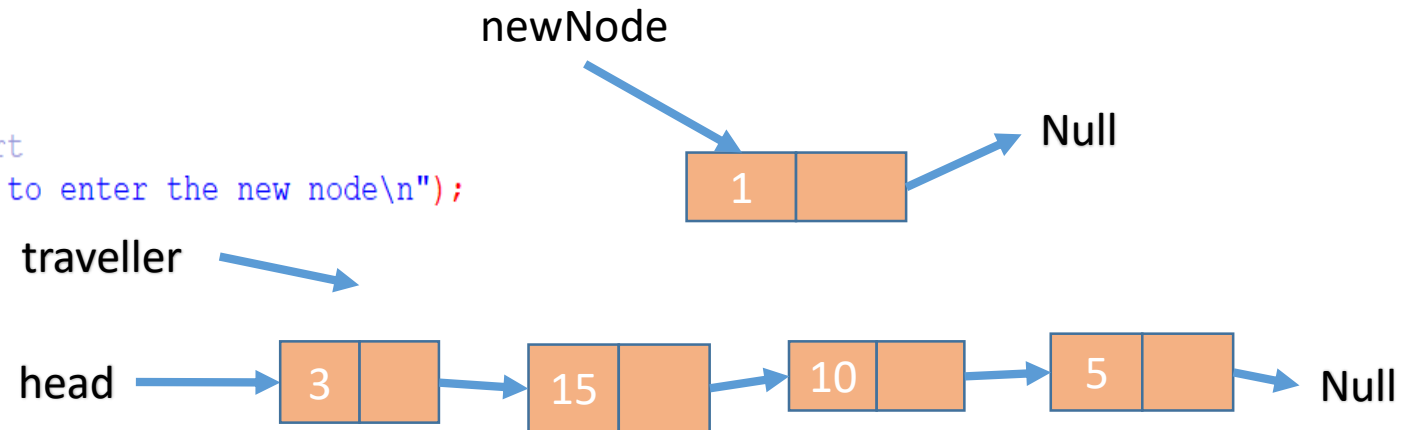


**Assume that we want to insert after 16
Clearly 16 does not exist**

Linked List Insertion (After a particular Node)

```
void addAfterAnElement() {
    if (head==NULL) {
        printf("The list is empty!");
        return;
    }
    // Making the new node by taking the node value from the user
    struct Node* newNode = malloc(sizeof(struct Node));
    printf("Enter the value of the new node\n");
    scanf("%d",&newNode->value);
    newNode->nextNode=NULL;
    // Taking the value of the node after which we will insert
    printf("Enter the value of the node after which you want to enter the new node\n");
    int valueOfThePredecessorNode;
    scanf("%d",&valueOfThePredecessorNode);
    // finding that particular node
    struct Node* traveller;
    traveller=head;
    while(traveller!=NULL) {
        if(traveller->value==valueOfThePredecessorNode) {
            newNode->nextNode=traveller->nextNode;
            traveller->nextNode=newNode;
            return;
        }
        traveller=traveller->nextNode;
    }
    printf("The node after which you wanted to insert does not exist!\n");
}
```

**CASE 3: LIST IS NOT EMPTY &
DOES NOT ELEMENT EXISTS**

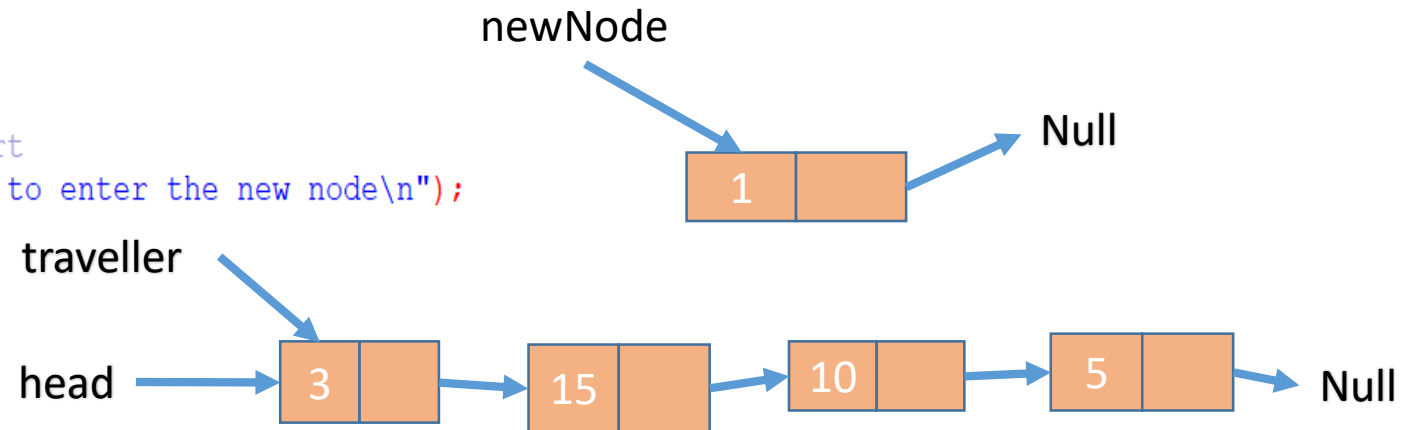


**Assume that we want to insert after 16
Clearly 16 does not exist**

Linked List Insertion (After a particular Node)

```
void addAfterAnElement() {
    if (head==NULL) {
        printf("The list is empty!");
        return;
    }
    // Making the new node by taking the node value from the user
    struct Node* newNode = malloc(sizeof(struct Node));
    printf("Enter the value of the new node\n");
    scanf("%d",&newNode->value);
    newNode->nextNode=NULL;
    // Taking the value of the node after which we will insert
    printf("Enter the value of the node after which you want to enter the new node\n");
    int valueOfThePredecessorNode;
    scanf("%d",&valueOfThePredecessorNode);
    // finding that particular node
    struct Node* traveller;
    traveller=head;
    while(traveller!=NULL) {
        if(traveller->value==valueOfThePredecessorNode) {
            newNode->nextNode=traveller->nextNode;
            traveller->nextNode=newNode;
            return;
        }
        traveller=traveller->nextNode;
    }
    printf("The node after which you wanted to insert does not exist!\n");
}
```

**CASE 3: LIST IS NOT EMPTY &
DOES NOT ELEMENT EXISTS**

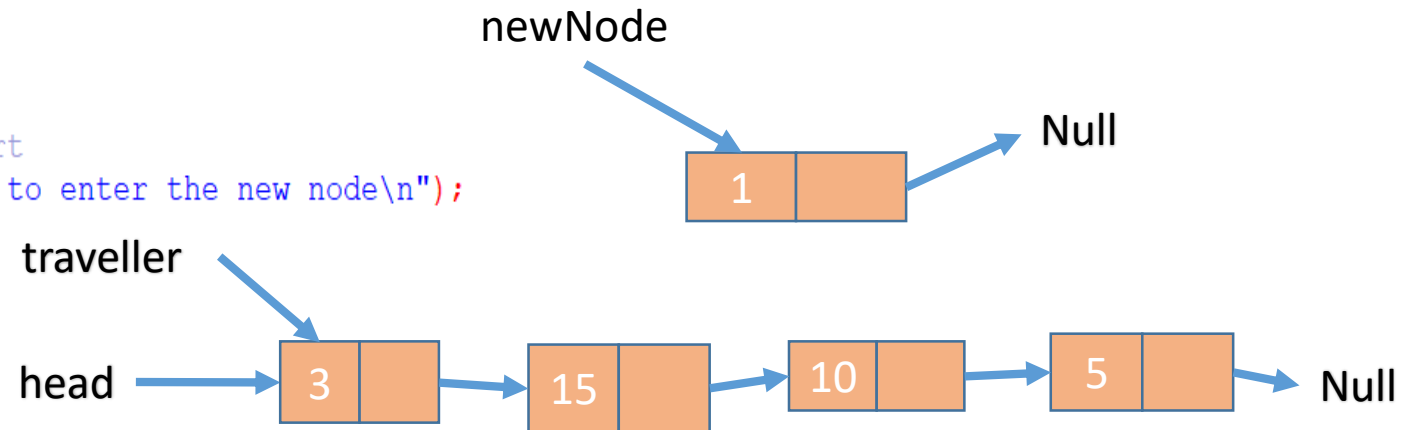


**Assume that we want to insert after 16
Clearly 16 does not exist**

Linked List Insertion (After a particular Node)

```
void addAfterAnElement() {
    if (head==NULL) {
        printf("The list is empty!");
        return;
    }
    // Making the new node by taking the node value from the user
    struct Node* newNode = malloc(sizeof(struct Node));
    printf("Enter the value of the new node\n");
    scanf("%d",&newNode->value);
    newNode->nextNode=NULL;
    // Taking the value of the node after which we will insert
    printf("Enter the value of the node after which you want to enter the new node\n");
    int valueOfThePredecessorNode;
    scanf("%d",&valueOfThePredecessorNode);
    // finding that particular node
    struct Node* traveller;
    traveller=head;
    while(traveller!=NULL) {
        if(traveller->value==valueOfThePredecessorNode) {
            newNode->nextNode=traveller->nextNode;
            traveller->nextNode=newNode;
            return;
        }
        traveller=traveller->nextNode;
    }
    printf("The node after which you wanted to insert does not exist!\n");
}
```

**CASE 3: LIST IS NOT EMPTY &
DOES NOT ELEMENT EXISTS**

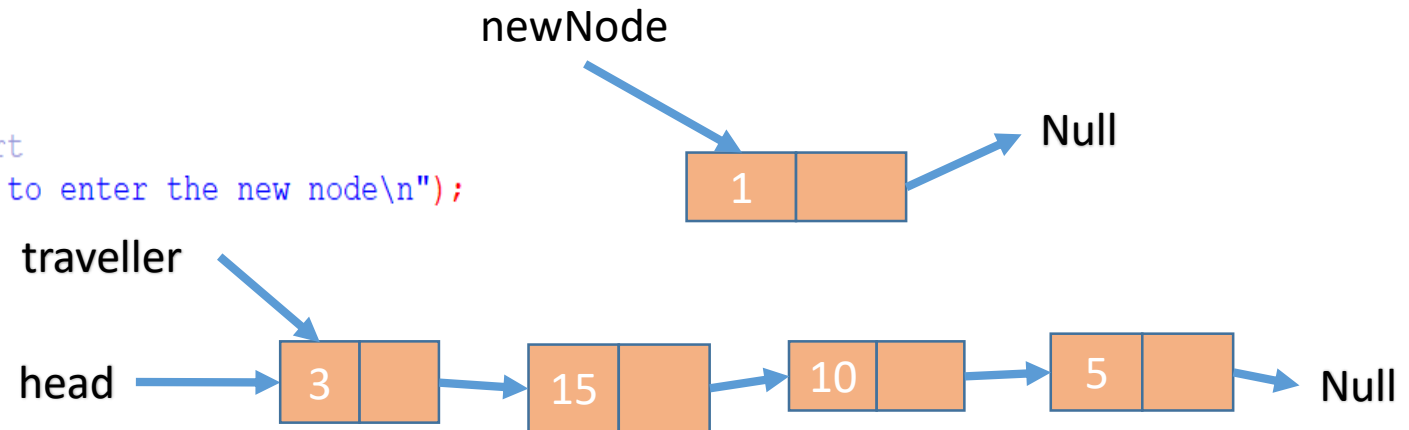


**Assume that we want to insert after 16
Clearly 16 does not exist**

Linked List Insertion (After a particular Node)

```
void addAfterAnElement() {
    if (head==NULL) {
        printf("The list is empty!");
        return;
    }
    // Making the new node by taking the node value from the user
    struct Node* newNode = malloc(sizeof(struct Node));
    printf("Enter the value of the new node\n");
    scanf("%d",&newNode->value);
    newNode->nextNode=NULL;
    // Taking the value of the node after which we will insert
    printf("Enter the value of the node after which you want to enter the new node\n");
    int valueOfThePredecessorNode;
    scanf("%d",&valueOfThePredecessorNode);
    // finding that particular node
    struct Node* traveller;
    traveller=head;
    while(traveller!=NULL) {
        if(traveller->value==valueOfThePredecessorNode) {
            newNode->nextNode=traveller->nextNode;
            traveller->nextNode=newNode;
            return;
        }
        traveller=traveller->nextNode;
    }
    printf("The node after which you wanted to insert does not exist!\n");
}
```

**CASE 3: LIST IS NOT EMPTY &
DOES NOT ELEMENT EXISTS**

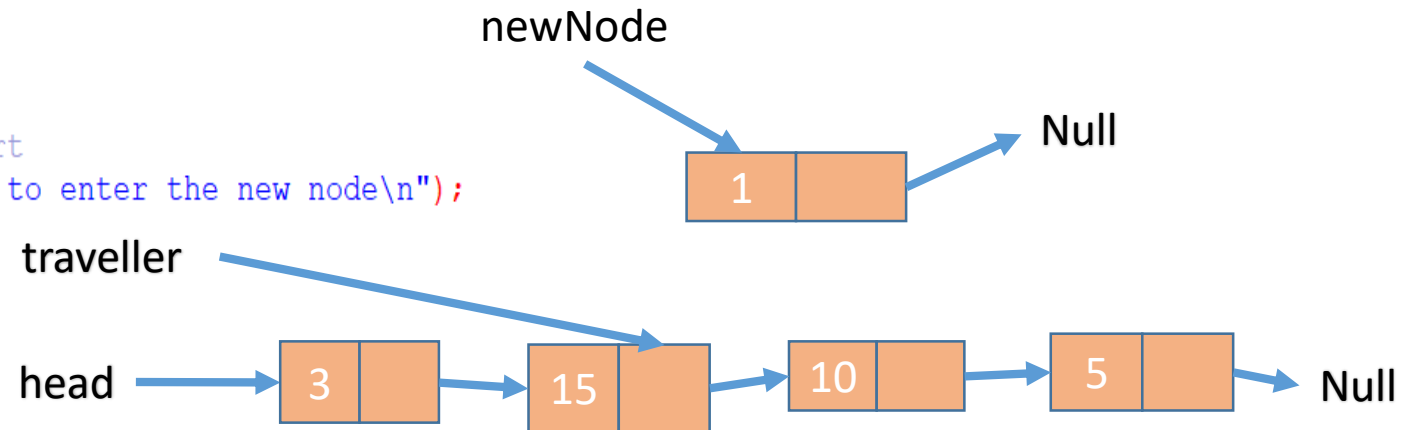


**Assume that we want to insert after 16
Clearly 16 does not exist**

Linked List Insertion (After a particular Node)

```
void addAfterAnElement() {
    if (head==NULL) {
        printf("The list is empty!");
        return;
    }
    // Making the new node by taking the node value from the user
    struct Node* newNode = malloc(sizeof(struct Node));
    printf("Enter the value of the new node\n");
    scanf("%d",&newNode->value);
    newNode->nextNode=NULL;
    // Taking the value of the node after which we will insert
    printf("Enter the value of the node after which you want to enter the new node\n");
    int valueOfThePredecessorNode;
    scanf("%d",&valueOfThePredecessorNode);
    // finding that particular node
    struct Node* traveller;
    traveller=head;
    while(traveller!=NULL) {
        if(traveller->value==valueOfThePredecessorNode) {
            newNode->nextNode=traveller->nextNode;
            traveller->nextNode=newNode;
            return;
        }
        traveller=traveller->nextNode;
    }
    printf("The node after which you wanted to insert does not exist!\n");
}
```

**CASE 3: LIST IS NOT EMPTY &
DOES NOT ELEMENT EXISTS**

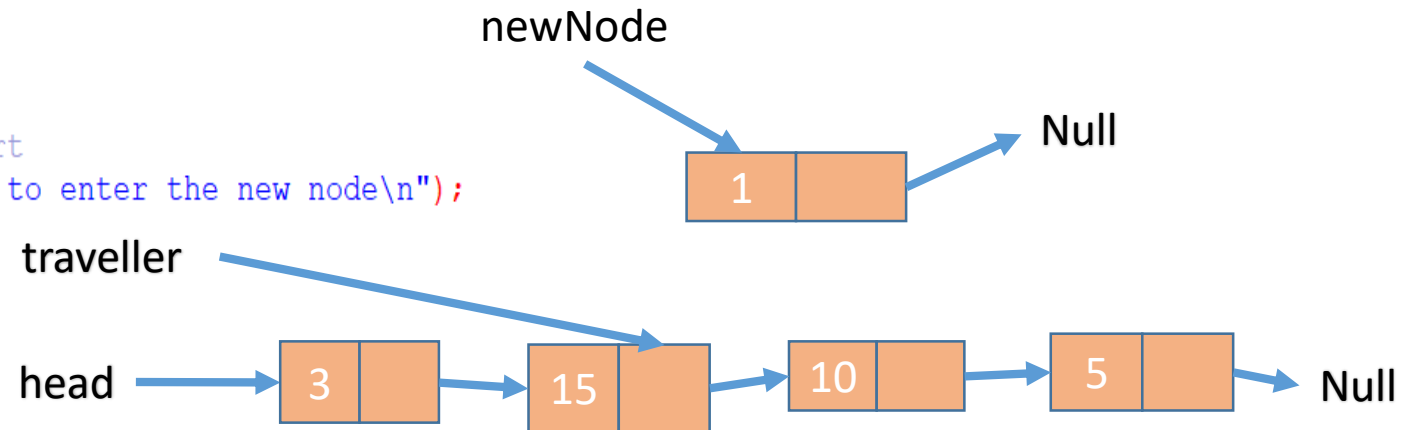


**Assume that we want to insert after 16
Clearly 16 does not exist**

Linked List Insertion (After a particular Node)

```
void addAfterAnElement() {
    if (head==NULL) {
        printf("The list is empty!");
        return;
    }
    // Making the new node by taking the node value from the user
    struct Node* newNode = malloc(sizeof(struct Node));
    printf("Enter the value of the new node\n");
    scanf("%d",&newNode->value);
    newNode->nextNode=NULL;
    // Taking the value of the node after which we will insert
    printf("Enter the value of the node after which you want to enter the new node\n");
    int valueOfThePredecessorNode;
    scanf("%d",&valueOfThePredecessorNode);
    // finding that particular node
    struct Node* traveller;
    traveller=head;
    while(traveller!=NULL) {
        if(traveller->value==valueOfThePredecessorNode) {
            newNode->nextNode=traveller->nextNode;
            traveller->nextNode=newNode;
            return;
        }
        traveller=traveller->nextNode;
    }
    printf("The node after which you wanted to insert does not exist!\n");
}
```

**CASE 3: LIST IS NOT EMPTY &
DOES NOT ELEMENT EXISTS**

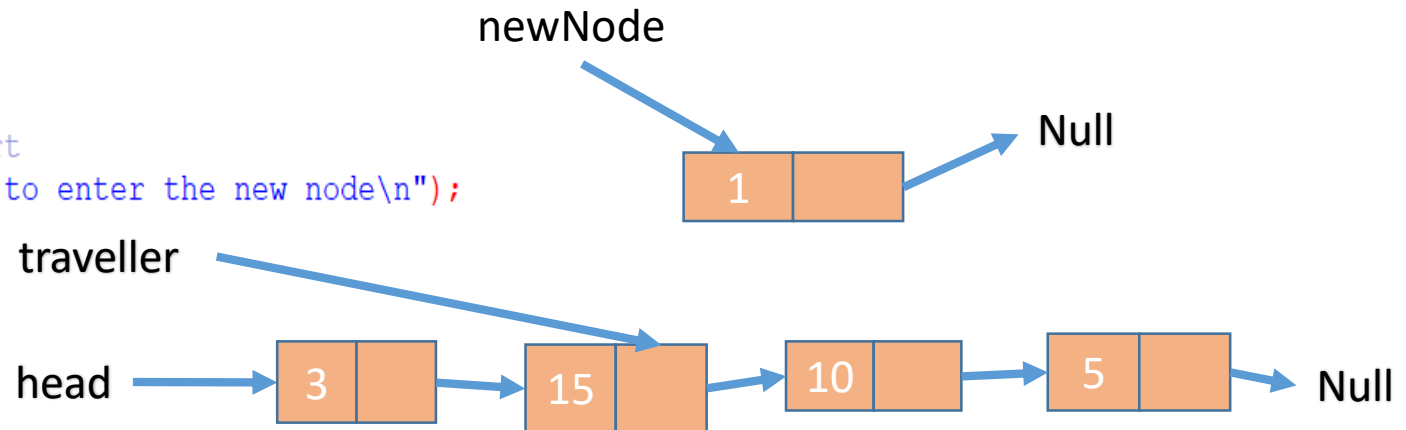


**Assume that we want to insert after 16
Clearly 16 does not exist**

Linked List Insertion (After a particular Node)

```
void addAfterAnElement() {
    if (head==NULL) {
        printf("The list is empty!");
        return;
    }
    // Making the new node by taking the node value from the user
    struct Node* newNode = malloc(sizeof(struct Node));
    printf("Enter the value of the new node\n");
    scanf("%d",&newNode->value);
    newNode->nextNode=NULL;
    // Taking the value of the node after which we will insert
    printf("Enter the value of the node after which you want to enter the new node\n");
    int valueOfThePredecessorNode;
    scanf("%d",&valueOfThePredecessorNode);
    // finding that particular node
    struct Node* traveller;
    traveller=head;
    while(traveller!=NULL) {
        if(traveller->value==valueOfThePredecessorNode) {
            newNode->nextNode=traveller->nextNode;
            traveller->nextNode=newNode;
            return;
        }
        traveller=traveller->nextNode;
    }
    printf("The node after which you wanted to insert does not exist!\n");
}
```

**CASE 3: LIST IS NOT EMPTY &
DOES NOT ELEMENT EXISTS**

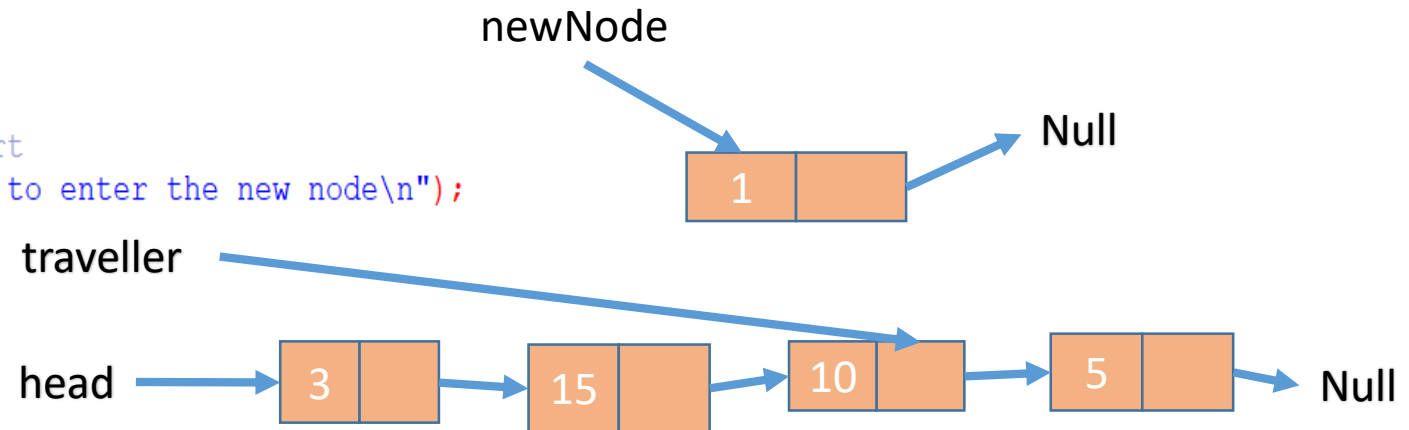


**Assume that we want to insert after 16
Clearly 16 does not exist**

Linked List Insertion (After a particular Node)

```
void addAfterAnElement() {
    if (head==NULL) {
        printf("The list is empty!");
        return;
    }
    // Making the new node by taking the node value from the user
    struct Node* newNode = malloc(sizeof(struct Node));
    printf("Enter the value of the new node\n");
    scanf("%d",&newNode->value);
    newNode->nextNode=NULL;
    // Taking the value of the node after which we will insert
    printf("Enter the value of the node after which you want to enter the new node\n");
    int valueOfThePredecessorNode;
    scanf("%d",&valueOfThePredecessorNode);
    // finding that particular node
    struct Node* traveller;
    traveller=head;
    while(traveller!=NULL) {
        if(traveller->value==valueOfThePredecessorNode) {
            newNode->nextNode=traveller->nextNode;
            traveller->nextNode=newNode;
            return;
        }
        traveller=traveller->nextNode;
    }
    printf("The node after which you wanted to insert does not exist!\n");
}
```

**CASE 3: LIST IS NOT EMPTY &
DOES NOT ELEMENT EXISTS**

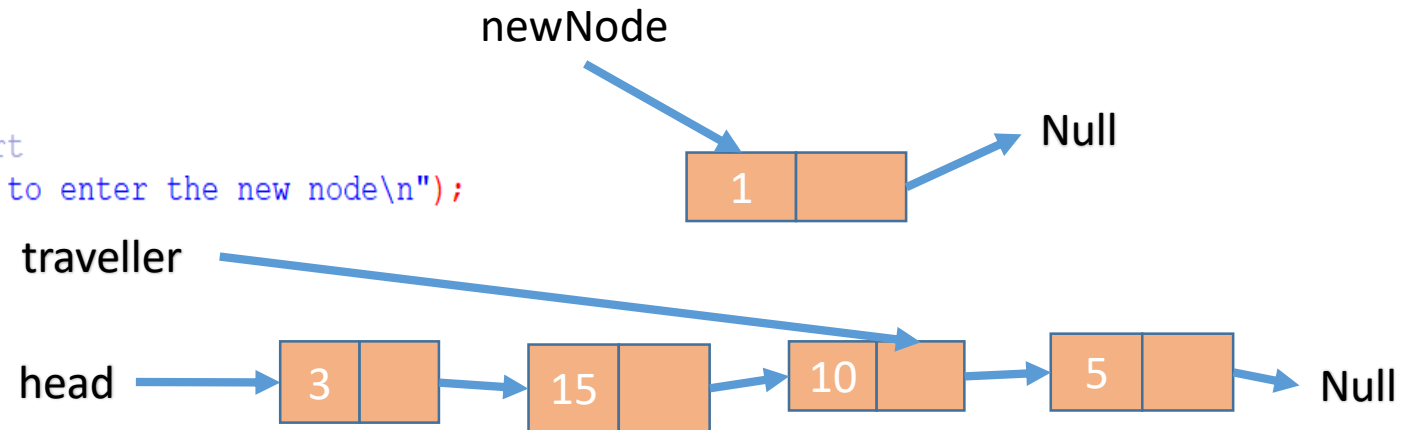


**Assume that we want to insert after 16
Clearly 16 does not exist**

Linked List Insertion (After a particular Node)

```
void addAfterAnElement() {
    if (head==NULL) {
        printf("The list is empty!");
        return;
    }
    // Making the new node by taking the node value from the user
    struct Node* newNode = malloc(sizeof(struct Node));
    printf("Enter the value of the new node\n");
    scanf("%d",&newNode->value);
    newNode->nextNode=NULL;
    // Taking the value of the node after which we will insert
    printf("Enter the value of the node after which you want to enter the new node\n");
    int valueOfThePredecessorNode;
    scanf("%d",&valueOfThePredecessorNode);
    // finding that particular node
    struct Node* traveller;
    traveller=head;
    while(traveller!=NULL) {
        if(traveller->value==valueOfThePredecessorNode) {
            newNode->nextNode=traveller->nextNode;
            traveller->nextNode=newNode;
            return;
        }
        traveller=traveller->nextNode;
    }
    printf("The node after which you wanted to insert does not exist!\n");
}
```

**CASE 3: LIST IS NOT EMPTY &
DOES NOT ELEMENT EXISTS**

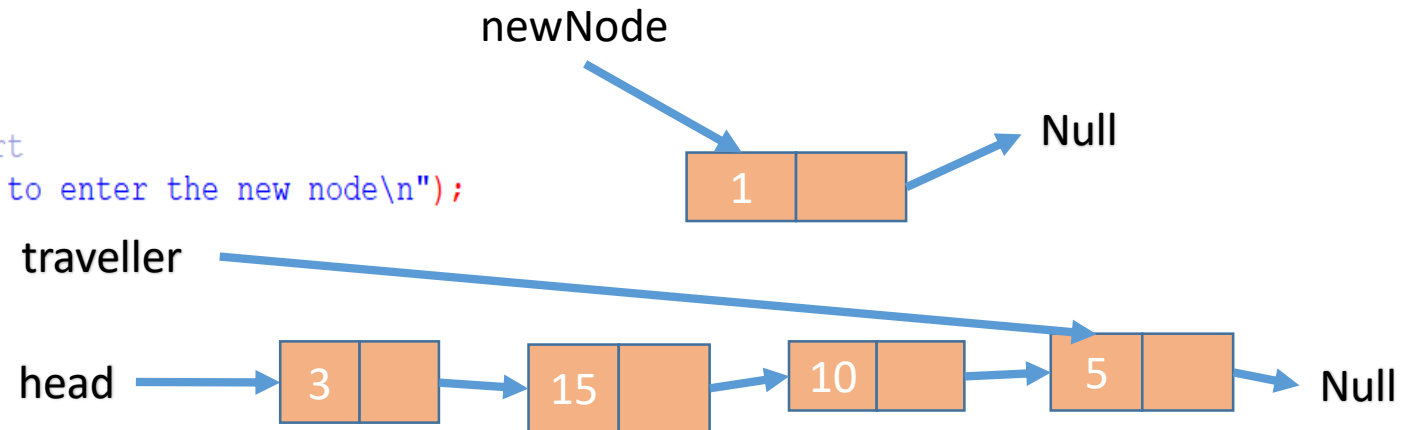


**Assume that we want to insert after 16
Clearly 16 does not exist**

Linked List Insertion (After a particular Node)

```
void addAfterAnElement() {
    if (head==NULL) {
        printf("The list is empty!");
        return;
    }
    // Making the new node by taking the node value from the user
    struct Node* newNode = malloc(sizeof(struct Node));
    printf("Enter the value of the new node\n");
    scanf("%d",&newNode->value);
    newNode->nextNode=NULL;
    // Taking the value of the node after which we will insert
    printf("Enter the value of the node after which you want to enter the new node\n");
    int valueOfThePredecessorNode;
    scanf("%d",&valueOfThePredecessorNode);
    // finding that particular node
    struct Node* traveller;
    traveller=head;
    while(traveller!=NULL) {
        if(traveller->value==valueOfThePredecessorNode) {
            newNode->nextNode=traveller->nextNode;
            traveller->nextNode=newNode;
            return;
        }
        traveller=traveller->nextNode;
    }
    printf("The node after which you wanted to insert does not exist!\n");
}
```

**CASE 3: LIST IS NOT EMPTY &
DOES NOT ELEMENT EXISTS**

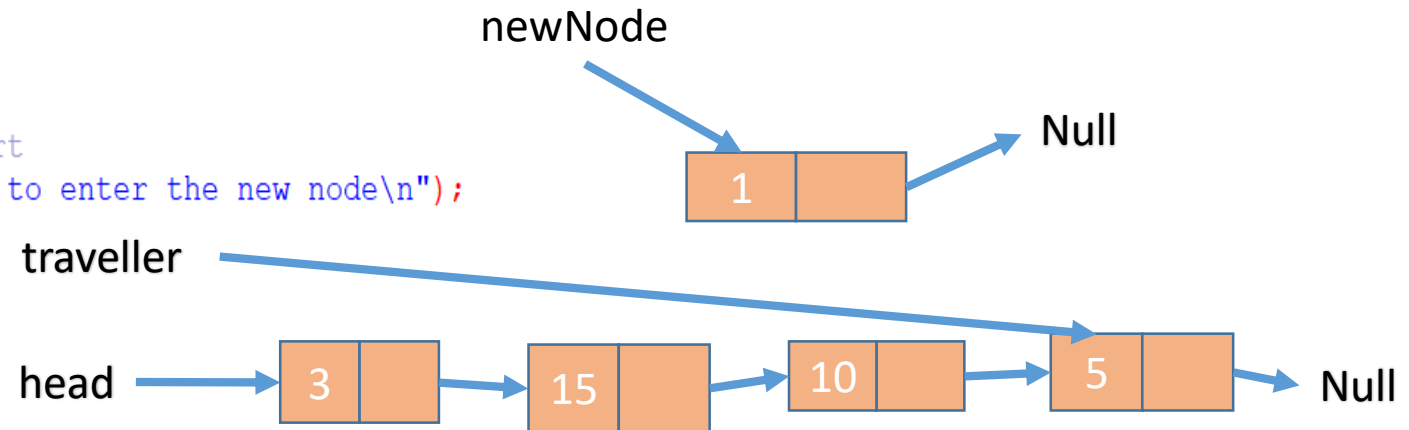


**Assume that we want to insert after 16
Clearly 16 does not exist**

Linked List Insertion (After a particular Node)

```
void addAfterAnElement() {
    if (head==NULL) {
        printf("The list is empty!");
        return;
    }
    // Making the new node by taking the node value from the user
    struct Node* newNode = malloc(sizeof(struct Node));
    printf("Enter the value of the new node\n");
    scanf("%d",&newNode->value);
    newNode->nextNode=NULL;
    // Taking the value of the node after which we will insert
    printf("Enter the value of the node after which you want to enter the new node\n");
    int valueOfThePredecessorNode;
    scanf("%d",&valueOfThePredecessorNode);
    // finding that particular node
    struct Node* traveller;
    traveller=head;
    while(traveller!=NULL) {
        if(traveller->value==valueOfThePredecessorNode) {
            newNode->nextNode=traveller->nextNode;
            traveller->nextNode=newNode;
            return;
        }
        traveller=traveller->nextNode;
    }
    printf("The node after which you wanted to insert does not exist!\n");
}
```

**CASE 3: LIST IS NOT EMPTY &
DOES NOT ELEMENT EXISTS**

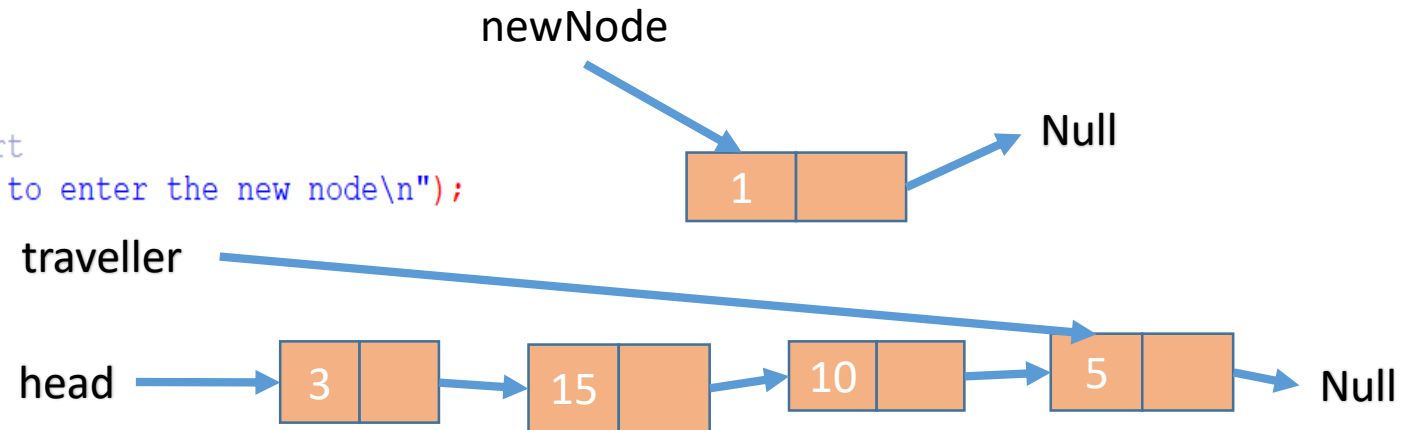


**Assume that we want to insert after 16
Clearly 16 does not exist**

Linked List Insertion (After a particular Node)

```
void addAfterAnElement() {
    if (head==NULL) {
        printf("The list is empty!");
        return;
    }
    // Making the new node by taking the node value from the user
    struct Node* newNode = malloc(sizeof(struct Node));
    printf("Enter the value of the new node\n");
    scanf("%d",&newNode->value);
    newNode->nextNode=NULL;
    // Taking the value of the node after which we will insert
    printf("Enter the value of the node after which you want to enter the new node\n");
    int valueOfThePredecessorNode;
    scanf("%d",&valueOfThePredecessorNode);
    // finding that particular node
    struct Node* traveller;
    traveller=head;
    while(traveller!=NULL) {
        if(traveller->value==valueOfThePredecessorNode) {
            newNode->nextNode=traveller->nextNode;
            traveller->nextNode=newNode;
            return;
        }
        traveller=traveller->nextNode;
    }
    printf("The node after which you wanted to insert does not exist!\n");
}
```

**CASE 3: LIST IS NOT EMPTY &
DOES NOT ELEMENT EXISTS**

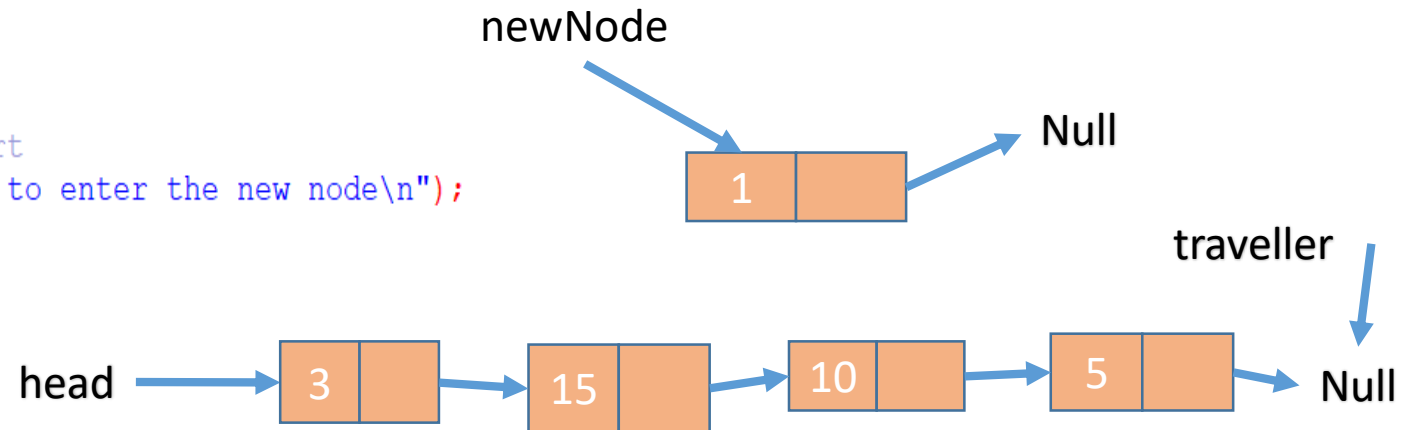


**Assume that we want to insert after 16
Clearly 16 does not exist**

Linked List Insertion (After a particular Node)

```
void addAfterAnElement() {
    if (head==NULL) {
        printf("The list is empty!");
        return;
    }
    // Making the new node by taking the node value from the user
    struct Node* newNode = malloc(sizeof(struct Node));
    printf("Enter the value of the new node\n");
    scanf("%d",&newNode->value);
    newNode->nextNode=NULL;
    // Taking the value of the node after which we will insert
    printf("Enter the value of the node after which you want to enter the new node\n");
    int valueOfThePredecessorNode;
    scanf("%d",&valueOfThePredecessorNode);
    // finding that particular node
    struct Node* traveller;
    traveller=head;
    while(traveller!=NULL) {
        if(traveller->value==valueOfThePredecessorNode) {
            newNode->nextNode=traveller->nextNode;
            traveller->nextNode=newNode;
            return;
        }
        traveller=traveller->nextNode;
    }
    printf("The node after which you wanted to insert does not exist!\n");
}
```

**CASE 3: LIST IS NOT EMPTY &
DOES NOT ELEMENT EXISTS**

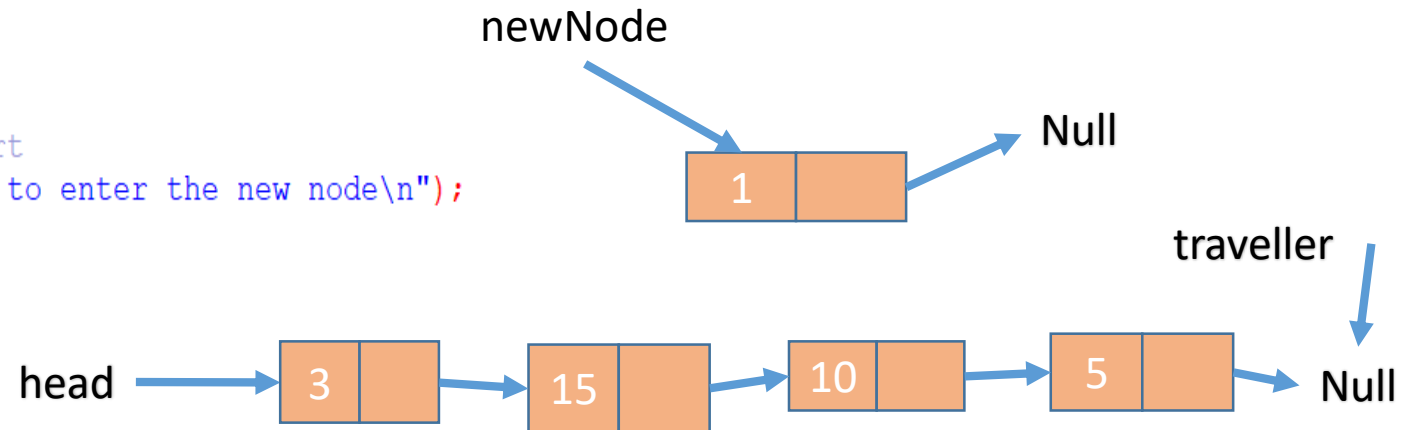


**Assume that we want to insert after 16
Clearly 16 does not exist**

Linked List Insertion (After a particular Node)

```
void addAfterAnElement() {
    if (head==NULL) {
        printf("The list is empty!");
        return;
    }
    // Making the new node by taking the node value from the user
    struct Node* newNode = malloc(sizeof(struct Node));
    printf("Enter the value of the new node\n");
    scanf("%d",&newNode->value);
    newNode->nextNode=NULL;
    // Taking the value of the node after which we will insert
    printf("Enter the value of the node after which you want to enter the new node\n");
    int valueOfThePredecessorNode;
    scanf("%d",&valueOfThePredecessorNode);
    // finding that particular node
    struct Node* traveller;
    traveller=head;
    while(traveller!=NULL) {
        if(traveller->value==valueOfThePredecessorNode) {
            newNode->nextNode=traveller->nextNode;
            traveller->nextNode=newNode;
            return;
        }
        traveller=traveller->nextNode;
    }
    printf("The node after which you wanted to insert does not exist!\n");
}
```

**CASE 3: LIST IS NOT EMPTY &
DOES NOT ELEMENT EXISTS**

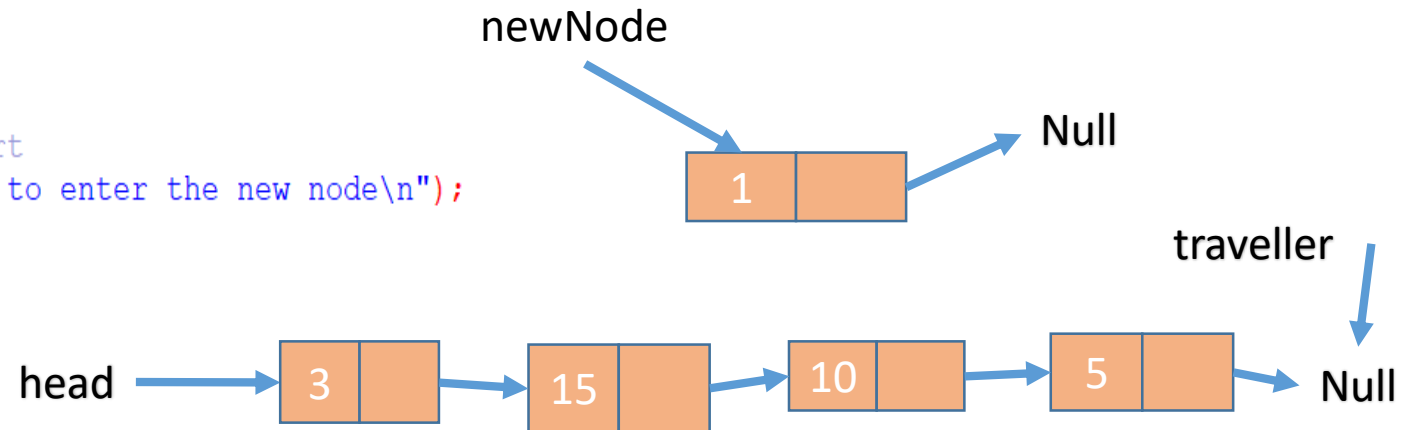


**Assume that we want to insert after 16
Clearly 16 does not exist**

Linked List Insertion (After a particular Node)

```
void addAfterAnElement() {
    if (head==NULL) {
        printf("The list is empty!");
        return;
    }
    // Making the new node by taking the node value from the user
    struct Node* newNode = malloc(sizeof(struct Node));
    printf("Enter the value of the new node\n");
    scanf("%d",&newNode->value);
    newNode->nextNode=NULL;
    // Taking the value of the node after which we will insert
    printf("Enter the value of the node after which you want to enter the new node\n");
    int valueOfThePredecessorNode;
    scanf("%d",&valueOfThePredecessorNode);
    // finding that particular node
    struct Node* traveller;
    traveller=head;
    while(traveller!=NULL) {
        if(traveller->value==valueOfThePredecessorNode) {
            newNode->nextNode=traveller->nextNode;
            traveller->nextNode=newNode;
            return;
        }
        traveller=traveller->nextNode;
    }
    printf("The node after which you wanted to insert does not exist!\n");
}
```

**CASE 3: LIST IS NOT EMPTY &
DOES NOT ELEMENT EXISTS**

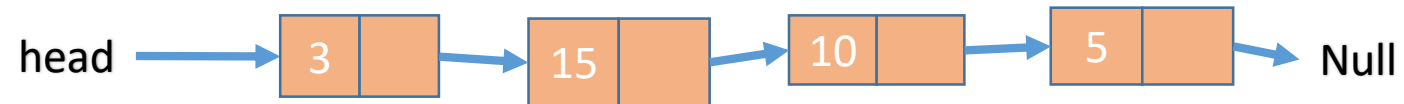


**Assume that we want to insert after 16
Clearly 16 does not exist**

Linked List Insertion (After a particular Node)

```
void addAfterAnElement() {
    if (head==NULL) {
        printf("The list is empty!");
        return;
    }
    // Making the new node by taking the node value from the user
    struct Node* newNode = malloc(sizeof(struct Node));
    printf("Enter the value of the new node\n");
    scanf("%d",&newNode->value);
    newNode->nextNode=NULL;
    // Taking the value of the node after which we will insert
    printf("Enter the value of the node after which you want to enter the new node\n");
    int valueOfThePredecessorNode;
    scanf("%d",&valueOfThePredecessorNode);
    // finding that particular node
    struct Node* traveller;
    traveller=head;
    while(traveller!=NULL) {
        if(traveller->value==valueOfThePredecessorNode) {
            newNode->nextNode=traveller->nextNode;
            traveller->nextNode=newNode;
            return;
        }
        traveller=traveller->nextNode;
    }
    printf("The node after which you wanted to insert does not exist!\n");
}
```

**CASE 3: LIST IS NOT EMPTY &
DOES NOT ELEMENT EXISTS**

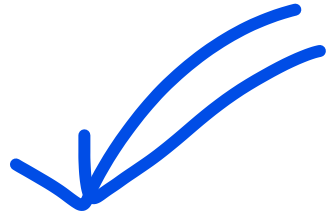




DELETION IN A LINKED LIST

Deletion at the beginning

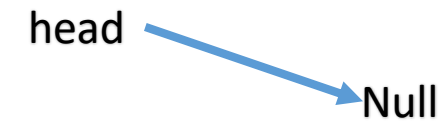
```
void deleteHead() {  
    if(head==NULL) {  
        printf("There is no list element to delete\n");  
        return;  
    }  
    struct Node *current;  
    current=head;  
    head=head->nextNode;  
    free(current) ;  
}
```



Deletion at the beginning

Case 1: The list is empty

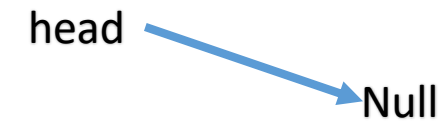
```
void deleteHead() {  
    if(head==NULL) {  
        printf("There is no list element to delete\n");  
        return;  
    }  
    struct Node *current;  
    current=head;  
    head=head->nextNode;  
    free(current) ;  
}
```



Deletion at the beginning

Case 1: The list is empty

```
void deleteHead() {  
    if (head == NULL) {  
        printf("There is no list element to delete\n");  
        return;  
    }  
    struct Node *current;  
    current = head;  
    head = head->nextNode;  
    free(current);  
}
```



Deletion at the beginning

Case 2: The list is NOT empty

```
void deleteHead() {  
    if(head==NULL) {  
        printf("There is no list element to delete\n");  
        return;  
    }  
    struct Node *current;  
    current=head;  
    head=head->nextNode;  
    free(current) ;  
}
```



Deletion at the beginning

Case 2: The list is NOT empty

```
void deleteHead() {  
    if(head==NULL) {  
        printf("There is no list element to delete\n");  
        return;  
    }  
    struct Node *current;  
    current=head;  
    head=head->nextNode;  
    free(current) ;  
}
```

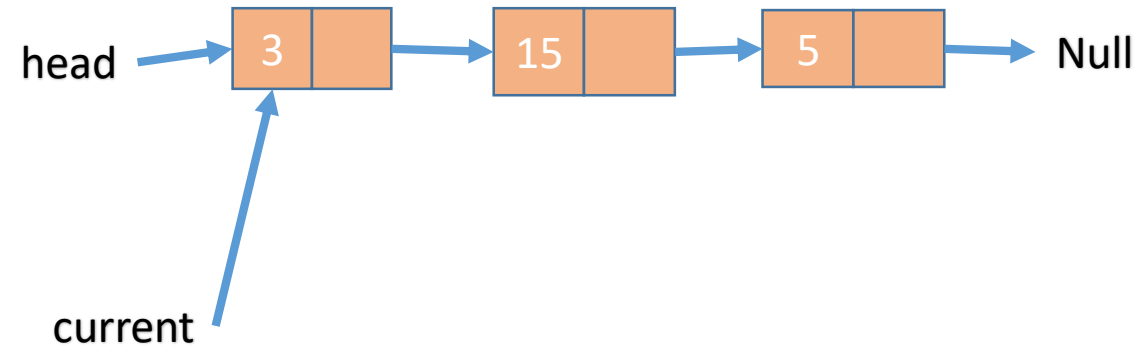


current →

Deletion at the beginning

Case 2: The list is NOT empty

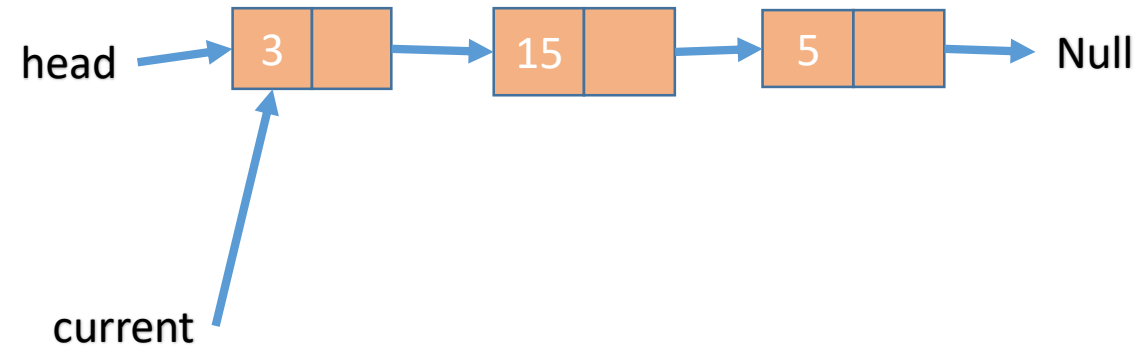
```
void deleteHead() {  
    if(head==NULL) {  
        printf("There is no list element to delete\n");  
        return;  
    }  
    struct Node *current;  
    → current=head;  
    head=head->nextNode;  
    free(current) ;  
}
```



Deletion at the beginning

Case 2: The list is NOT empty

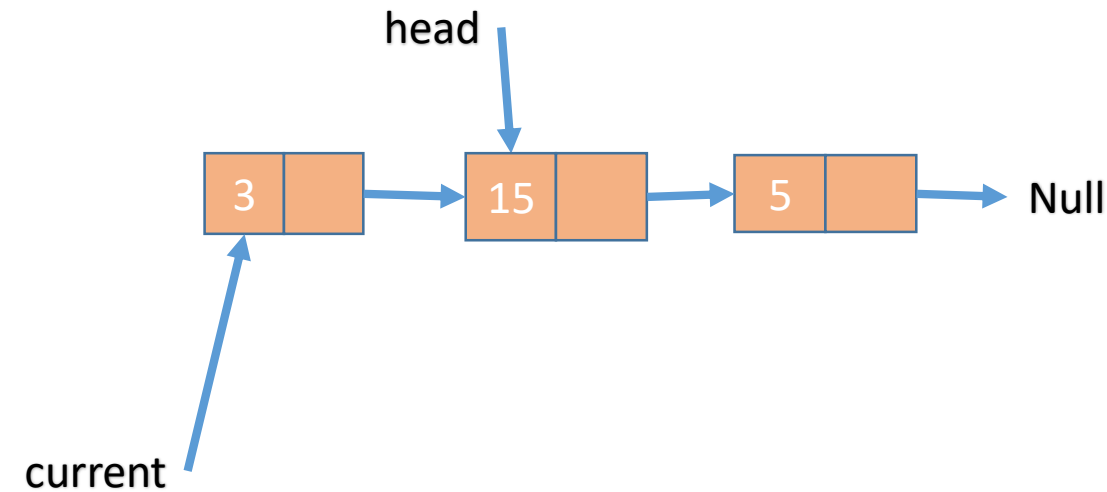
```
void deleteHead() {  
    if(head==NULL) {  
        printf("There is no list element to delete\n");  
        return;  
    }  
    struct Node *current;  
    current=head;  
    head=head->nextNode;  
    free(current) ;  
}
```



Deletion at the beginning

Case 2: The list is NOT empty

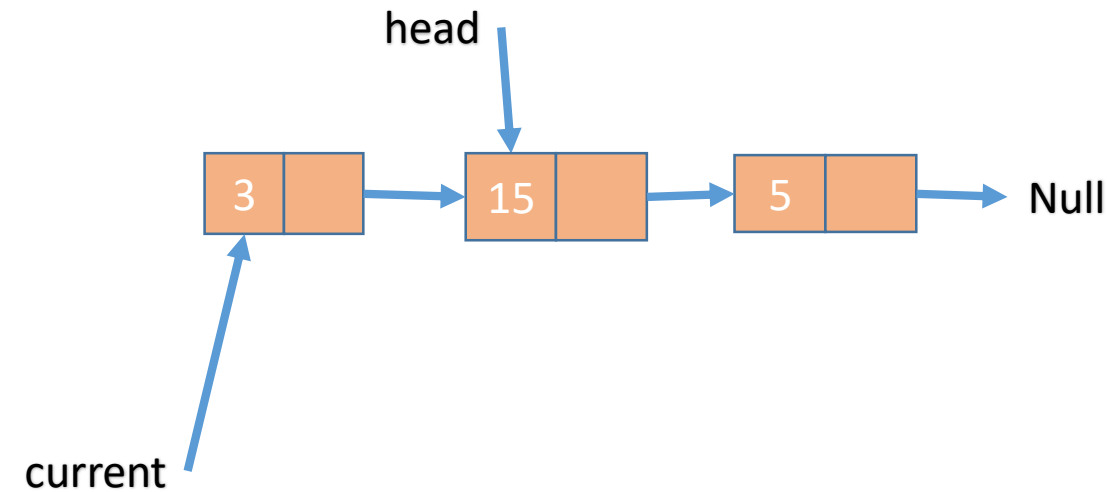
```
void deleteHead() {  
    if(head==NULL) {  
        printf("There is no list element to delete\n");  
        return;  
    }  
    struct Node *current;  
    current=head;  
    head=head->nextNode;  
    free(current) ;  
}
```



Deletion at the beginning

Case 2: The list is NOT empty

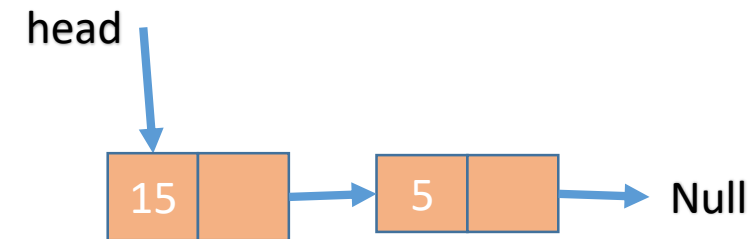
```
void deleteHead() {  
    if(head==NULL) {  
        printf("There is no list element to delete\n");  
        return;  
    }  
    struct Node *current;  
    current=head;  
    head=head->nextNode;  
    free(current) ;  
}
```



Deletion at the beginning

Case 2: The list is NOT empty

```
void deleteHead() {  
    if(head==NULL) {  
        printf("There is no list element to delete\n");  
        return;  
    }  
    struct Node *current;  
    current=head;  
    head=head->nextNode;  
    free(current) ;  
}
```

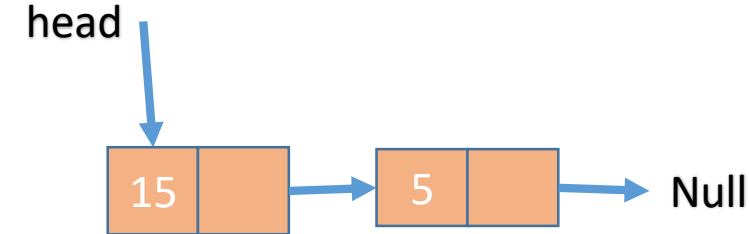


current

Deletion at the beginning

Case 2: The list is NOT empty

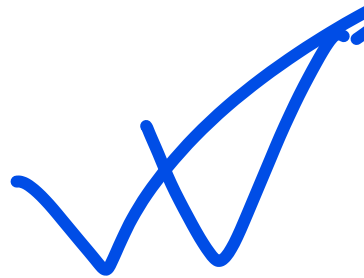
```
void deleteHead() {  
    if(head==NULL) {  
        printf("There is no list element to delete\n");  
        return;  
    }  
    struct Node *current;  
    current=head;  
    head=head->nextNode;  
    free(current) ;  
}
```



Deletion at the End

```
void deleteTail() {  
    if(head==NULL) {  
        printf("There is no list element to delete\n");  
        return;  
    }  
    struct Node *traveller=head;  
    struct Node *follower=NULL;  
  
    while(traveller->nextNode!=NULL) {  
        follower=traveller;  
        traveller=traveller->nextNode;  
    }  
  
    if(follower!=NULL) {  
        follower->nextNode=NULL;  
    } else {  
        head=NULL;  
    }  
  
    free(traveller);  
}
```

Case 1: The list is empty



Deletion at the End

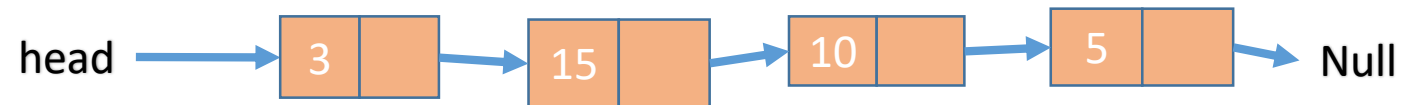
```
void deleteTail() {  
    if(head==NULL) {  
        printf("There is no list element to delete\n");  
        return;  
    }  
    struct Node *traveller=head;  
    struct Node *follower=NULL;  
  
    while(traveller->nextNode!=NULL) {  
        follower=traveller;  
        traveller=traveller->nextNode;  
    }  
  
    if(follower!=NULL) {  
        follower->nextNode=NULL;  
    } else {  
        head=NULL;  
    }  
  
    free(traveller);  
}
```

Case 2: The list is not empty

Deletion at the End

```
void deleteTail() {  
    if(head==NULL) {  
        printf("There is no list element to delete\n");  
        return;  
    }  
    struct Node *traveller=head;  
    struct Node *follower=NULL;  
  
    while(traveller->nextNode!=NULL) {  
        follower=traveller;  
        traveller=traveller->nextNode;  
    }  
  
    if(follower!=NULL) {  
        follower->nextNode=NULL;  
    } else {  
        head=NULL;  
    }  
  
    free(traveller);  
}
```

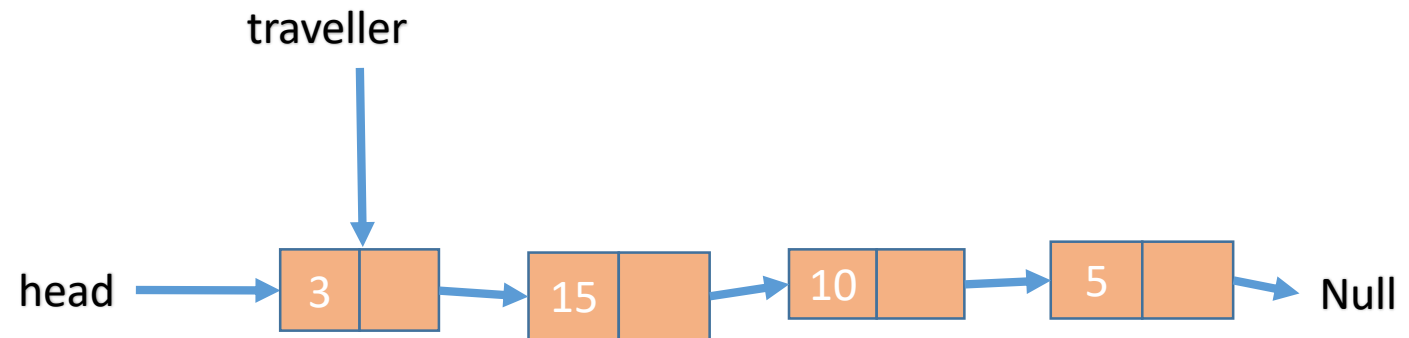
Case 2: The list is not empty
And there are many
Elements in the list



Deletion at the End

```
void deleteTail() {  
    if(head==NULL) {  
        printf("There is no list element to delete\n");  
        return;  
    }  
    struct Node *traveller=head;  
    struct Node *follower=NULL;  
  
    while(traveller->nextNode!=NULL) {  
        follower=traveller;  
        traveller=traveller->nextNode;  
    }  
  
    if(follower!=NULL) {  
        follower->nextNode=NULL;  
    } else {  
        head=NULL;  
    }  
  
    free(traveller);  
}
```

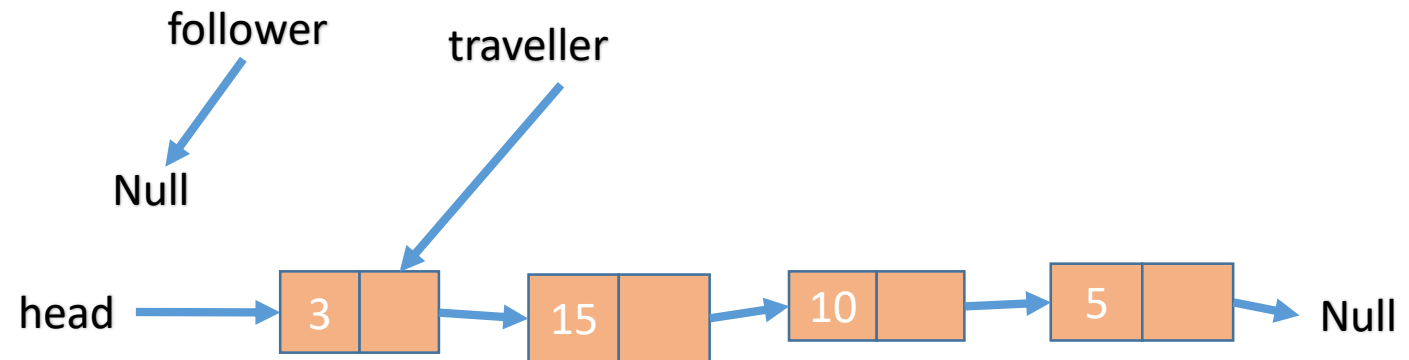
Case 2: The list is not empty
And there are many
Elements in the list



Deletion at the End

```
void deleteTail() {  
    if(head==NULL) {  
        printf("There is no list element to delete\n");  
        return;  
    }  
    struct Node *traveller=head;  
    struct Node *follower=NULL;  
  
    while(traveller->nextNode!=NULL) {  
        follower=traveller;  
        traveller=traveller->nextNode;  
    }  
  
    if(follower!=NULL) {  
        follower->nextNode=NULL;  
    } else {  
        head=NULL;  
    }  
  
    free(traveller);  
}
```

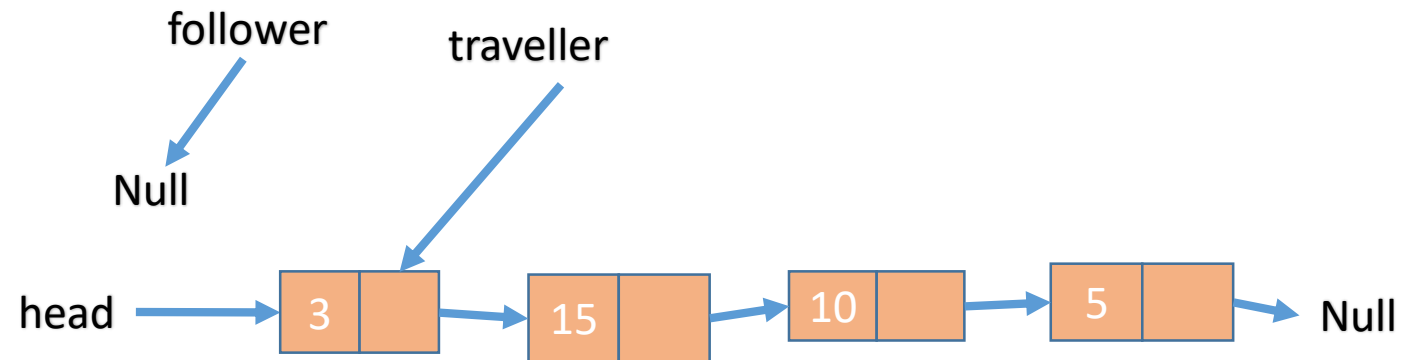
Case 2: The list is not empty
And there are many
Elements in the list



Deletion at the End

```
void deleteTail() {  
    if(head==NULL) {  
        printf("There is no list element to delete\n");  
        return;  
    }  
    struct Node *traveller=head;  
    struct Node *follower=NULL;  
    → while(traveller->nextNode!=NULL) {  
        follower=traveller;  
        traveller=traveller->nextNode;  
    }  
    if(follower!=NULL) {  
        follower->nextNode=NULL;  
    } else {  
        head=NULL;  
    }  
    free(traveller);  
}
```

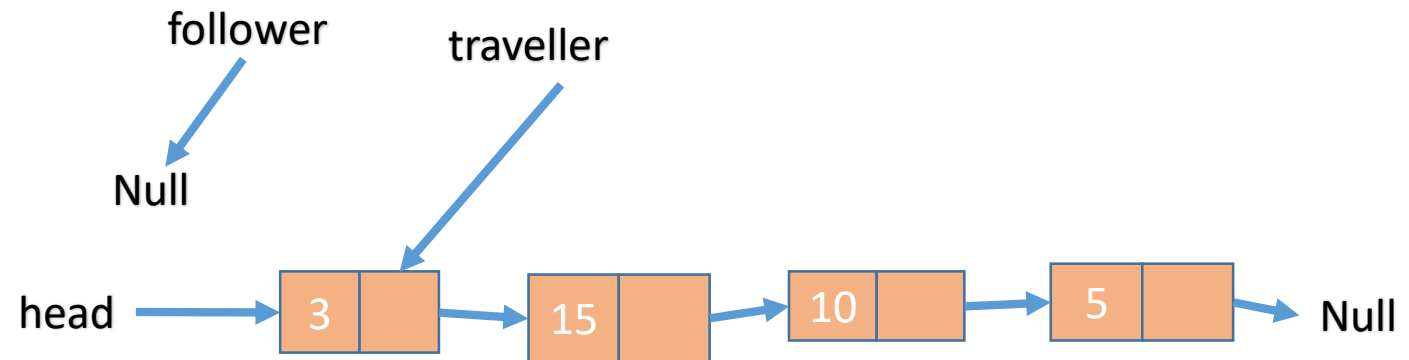
Case 2: The list is not empty
And there are many
Elements in the list



Deletion at the End

```
void deleteTail() {  
    if(head==NULL) {  
        printf("There is no list element to delete\n");  
        return;  
    }  
    struct Node *traveller=head;  
    struct Node *follower=NULL;  
  
    while(traveller->nextNode!=NULL) {  
        follower=traveller;  
        traveller=traveller->nextNode;  
    }  
  
    if(follower!=NULL) {  
        follower->nextNode=NULL;  
    } else {  
        head=NULL;  
    }  
  
    free(traveller);  
}
```

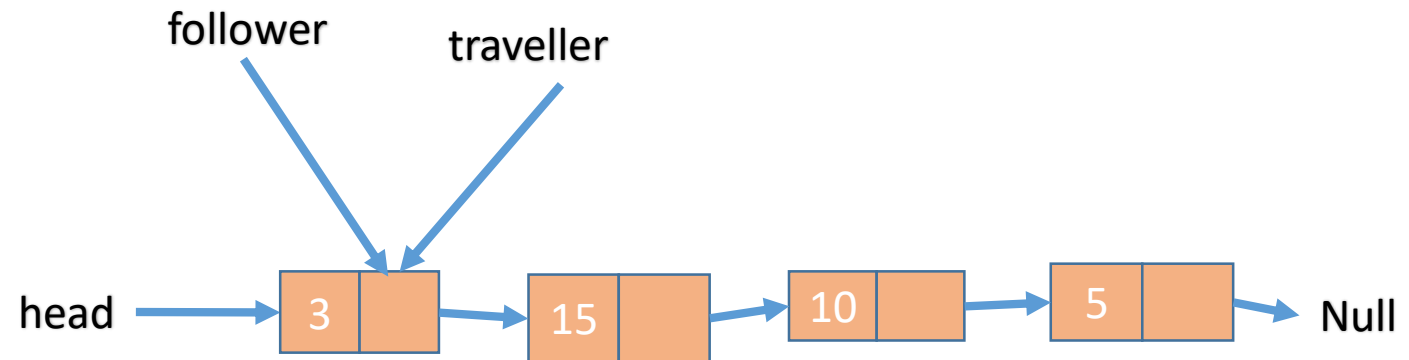
Case 2: The list is not empty
And there are many
Elements in the list



Deletion at the End

```
void deleteTail() {  
    if(head==NULL) {  
        printf("There is no list element to delete\n");  
        return;  
    }  
    struct Node *traveller=head;  
    struct Node *follower=NULL;  
  
    while(traveller->nextNode!=NULL) {  
        follower=traveller;  
        traveller=traveller->nextNode;  
    }  
  
    if(follower!=NULL) {  
        follower->nextNode=NULL;  
    } else {  
        head=NULL;  
    }  
  
    free(traveller);  
}
```

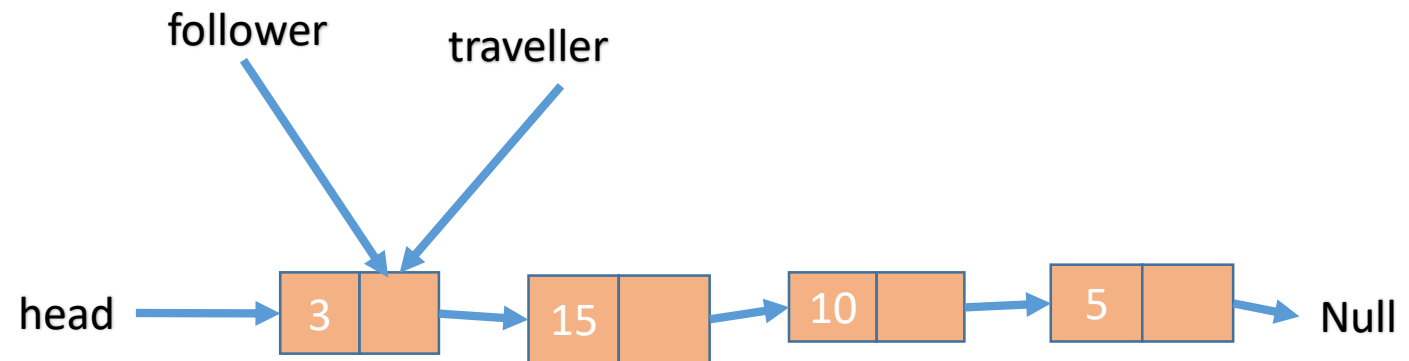
Case 2: The list is not empty
And there are many
Elements in the list



Deletion at the End

```
void deleteTail() {  
    if(head==NULL) {  
        printf("There is no list element to delete\n");  
        return;  
    }  
    struct Node *traveller=head;  
    struct Node *follower=NULL;  
  
    while(traveller->nextNode!=NULL) {  
        follower=traveller;  
        → traveller=traveller->nextNode;  
    }  
  
    if(follower!=NULL) {  
        follower->nextNode=NULL;  
    } else {  
        head=NULL;  
    }  
  
    free(traveller);  
}
```

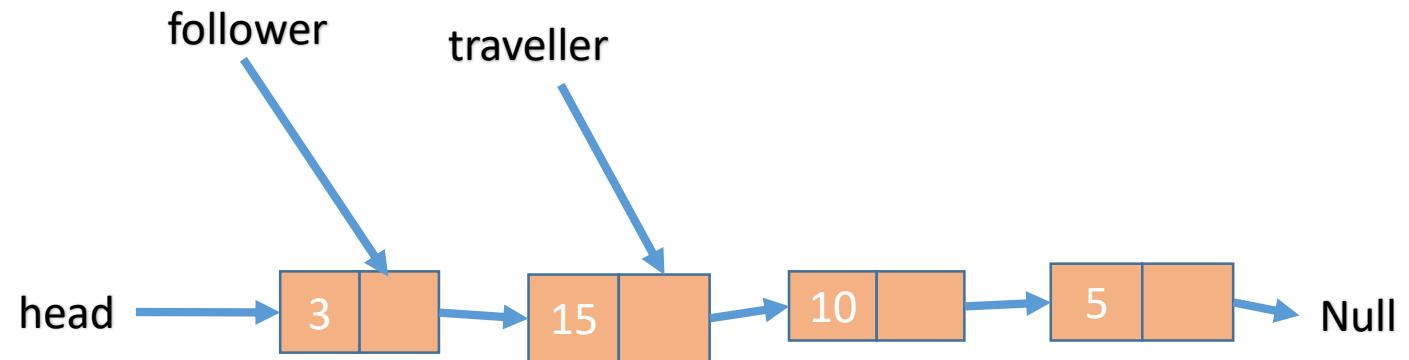
Case 2: The list is not empty
And there are many
Elements in the list



Deletion at the End

```
void deleteTail() {  
    if(head==NULL) {  
        printf("There is no list element to delete\n");  
        return;  
    }  
    struct Node *traveller=head;  
    struct Node *follower=NULL;  
  
    while(traveller->nextNode!=NULL) {  
        follower=traveller;  
        → traveller=traveller->nextNode;  
    }  
  
    if(follower!=NULL) {  
        follower->nextNode=NULL;  
    } else {  
        head=NULL;  
    }  
  
    free(traveller);  
}
```

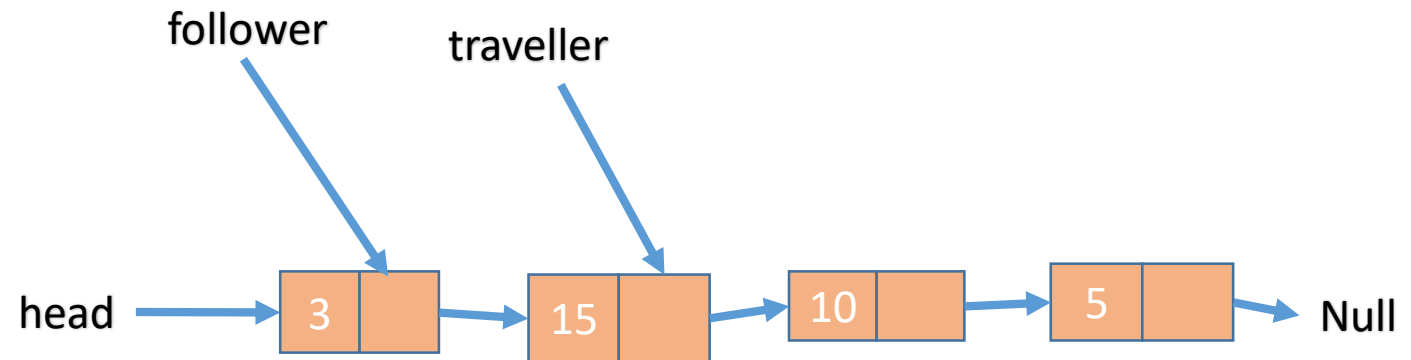
Case 2: The list is not empty
And there are many
Elements in the list



Deletion at the End

```
void deleteTail() {  
    if(head==NULL) {  
        printf("There is no list element to delete\n");  
        return;  
    }  
    struct Node *traveller=head;  
    struct Node *follower=NULL;  
    while(traveller->nextNode!=NULL) {  
        follower=traveller;  
        traveller=traveller->nextNode;  
    }  
    if(follower!=NULL) {  
        follower->nextNode=NULL;  
    } else {  
        head=NULL;  
    }  
    free(traveller);  
}
```

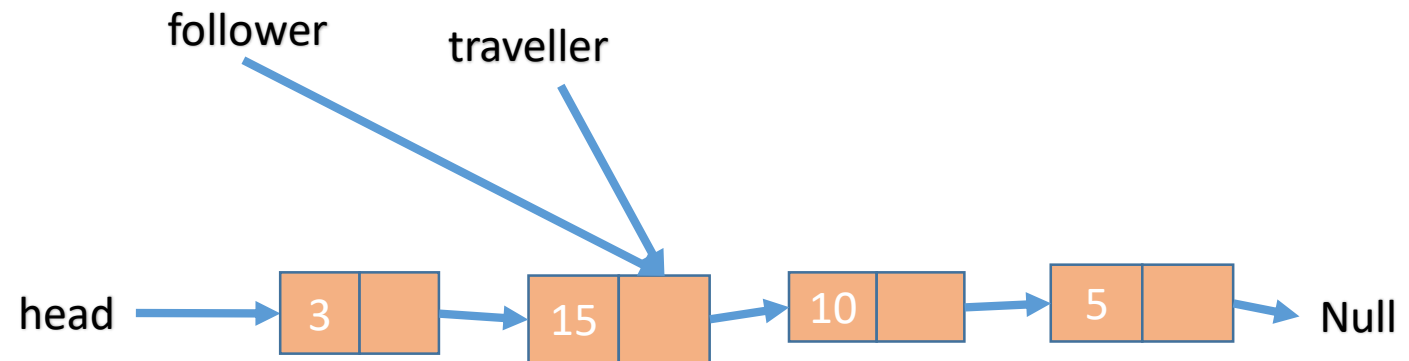
Case 2: The list is not empty
And there are many
Elements in the list



Deletion at the End

```
void deleteTail() {  
    if(head==NULL) {  
        printf("There is no list element to delete\n");  
        return;  
    }  
    struct Node *traveller=head;  
    struct Node *follower=NULL;  
  
    while(traveller->nextNode!=NULL) {  
        → follower=traveller;  
        traveller=traveller->nextNode;  
    }  
  
    if(follower!=NULL) {  
        follower->nextNode=NULL;  
    } else {  
        head=NULL;  
    }  
  
    free(traveller);  
}
```

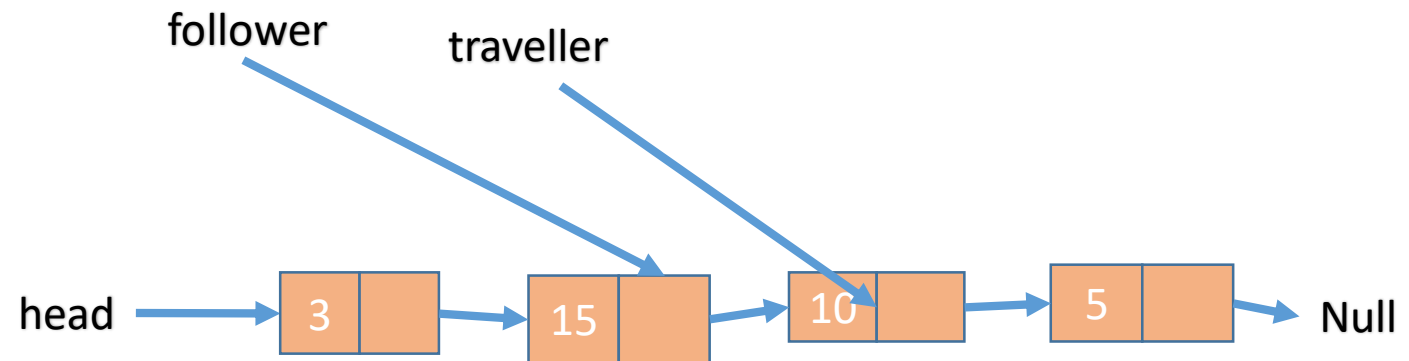
Case 2: The list is not empty
And there are many
Elements in the list



Deletion at the End

```
void deleteTail() {  
    if(head==NULL) {  
        printf("There is no list element to delete\n");  
        return;  
    }  
    struct Node *traveller=head;  
    struct Node *follower=NULL;  
  
    while(traveller->nextNode!=NULL) {  
        follower=traveller;  
        traveller=traveller->nextNode;  
    }  
  
    if(follower!=NULL) {  
        follower->nextNode=NULL;  
    } else {  
        head=NULL;  
    }  
  
    free(traveller);  
}
```

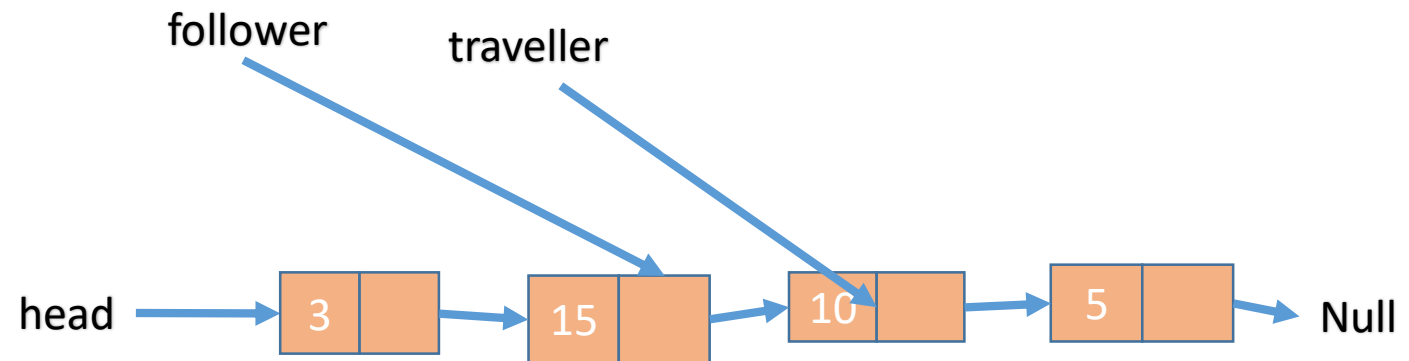
Case 2: The list is not empty
And there are many
Elements in the list



Deletion at the End

```
void deleteTail() {  
    if(head==NULL) {  
        printf("There is no list element to delete\n");  
        return;  
    }  
    struct Node *traveller=head;  
    struct Node *follower=NULL;  
    while(traveller->nextNode!=NULL) {  
        follower=traveller;  
        traveller=traveller->nextNode;  
    }  
    if(follower!=NULL) {  
        follower->nextNode=NULL;  
    } else {  
        head=NULL;  
    }  
    free(traveller);  
}
```

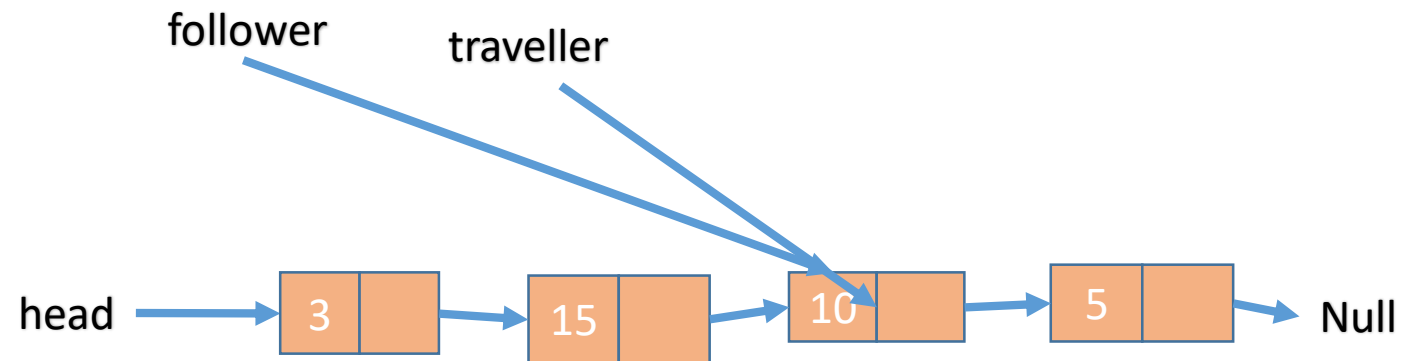
Case 2: The list is not empty
And there are many
Elements in the list



Deletion at the End

```
void deleteTail() {  
    if(head==NULL) {  
        printf("There is no list element to delete\n");  
        return;  
    }  
    struct Node *traveller=head;  
    struct Node *follower=NULL;  
  
    while(traveller->nextNode!=NULL) {  
        follower=traveller;  
        traveller=traveller->nextNode;  
    }  
  
    if(follower!=NULL) {  
        follower->nextNode=NULL;  
    } else {  
        head=NULL;  
    }  
  
    free(traveller);  
}
```

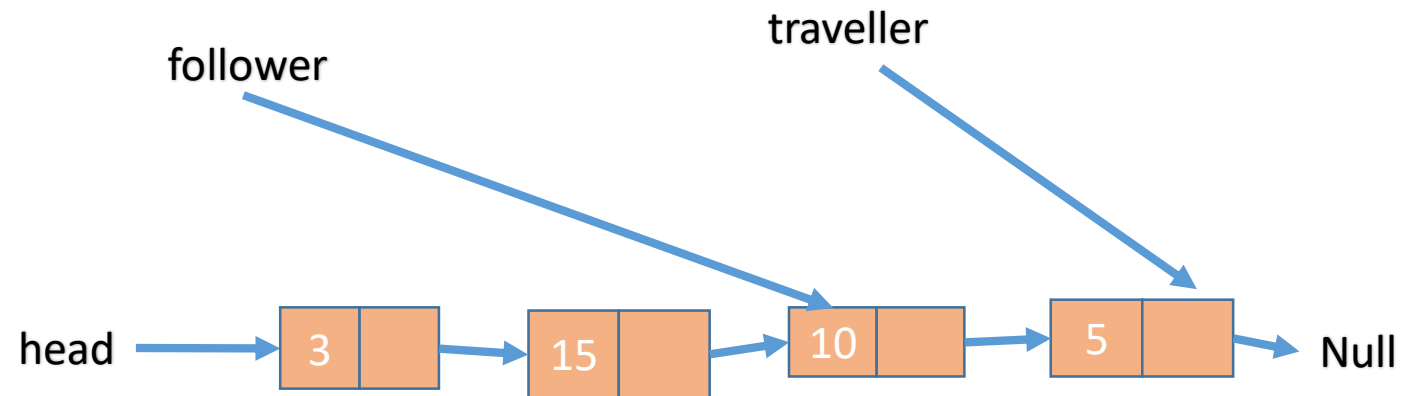
Case 2: The list is not empty
And there are many
Elements in the list



Deletion at the End

```
void deleteTail() {  
    if(head==NULL) {  
        printf("There is no list element to delete\n");  
        return;  
    }  
    struct Node *traveller=head;  
    struct Node *follower=NULL;  
  
    while(traveller->nextNode!=NULL) {  
        follower=traveller;  
        traveller=traveller->nextNode;  
    }  
  
    if(follower!=NULL) {  
        follower->nextNode=NULL;  
    } else {  
        head=NULL;  
    }  
  
    free(traveller);  
}
```

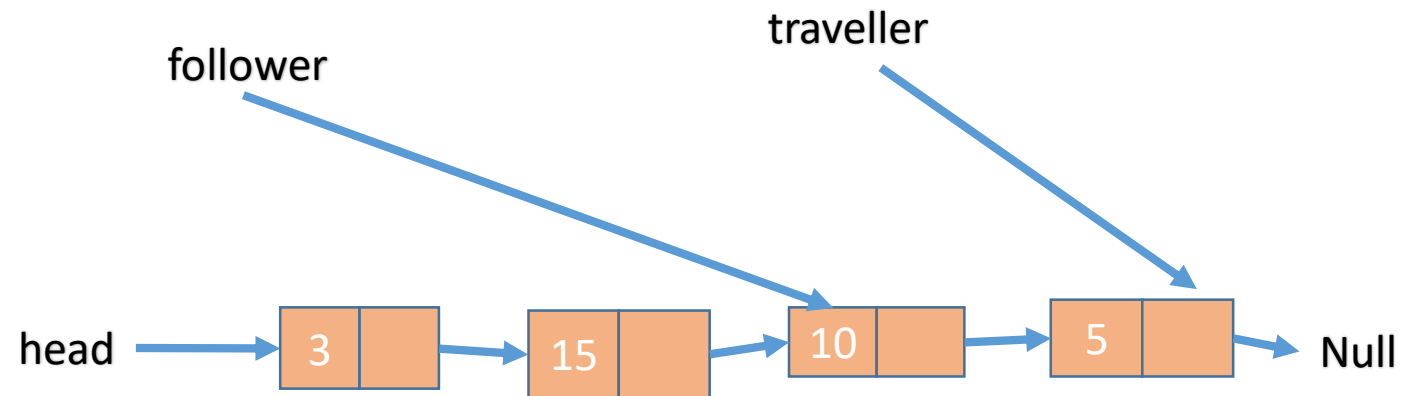
Case 2: The list is not empty
And there are many
Elements in the list



Deletion at the End

```
void deleteTail() {  
    if(head==NULL) {  
        printf("There is no list element to delete\n");  
        return;  
    }  
    struct Node *traveller=head;  
    struct Node *follower=NULL;  
    while(traveller->nextNode!=NULL) {  
        follower=traveller;  
        traveller=traveller->nextNode;  
    }  
    if(follower!=NULL) {  
        follower->nextNode=NULL;  
    } else {  
        head=NULL;  
    }  
    free(traveller);  
}
```

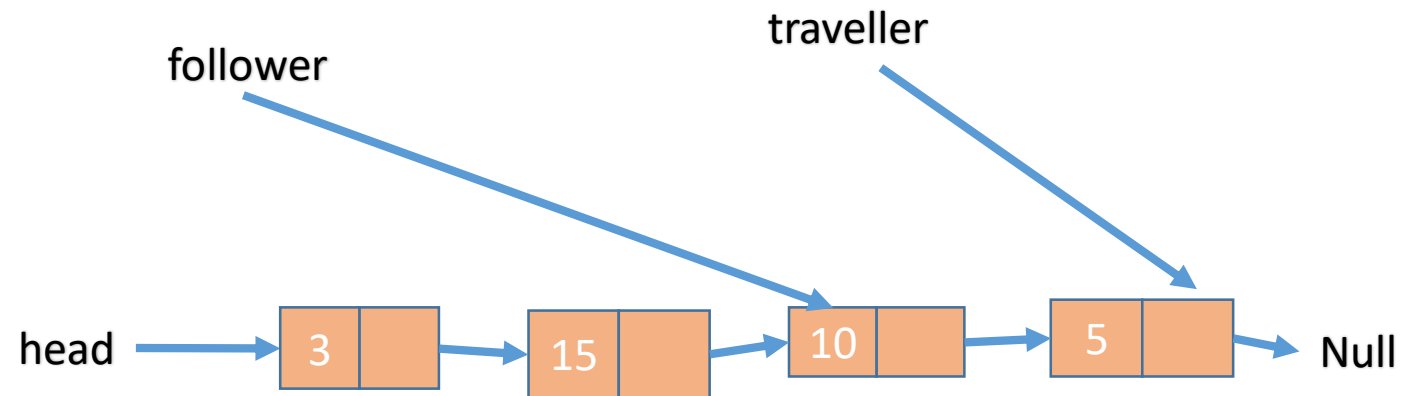
Case 2: The list is not empty
And there are many
Elements in the list



Deletion at the End

```
void deleteTail() {  
    if(head==NULL) {  
        printf("There is no list element to delete\n");  
        return;  
    }  
    struct Node *traveller=head;  
    struct Node *follower=NULL;  
  
    while(traveller->nextNode!=NULL) {  
        follower=traveller;  
        traveller=traveller->nextNode;  
    }  
    if(follower!=NULL) {  
        follower->nextNode=NULL;  
    } else {  
        head=NULL;  
    }  
  
    free(traveller);  
}
```

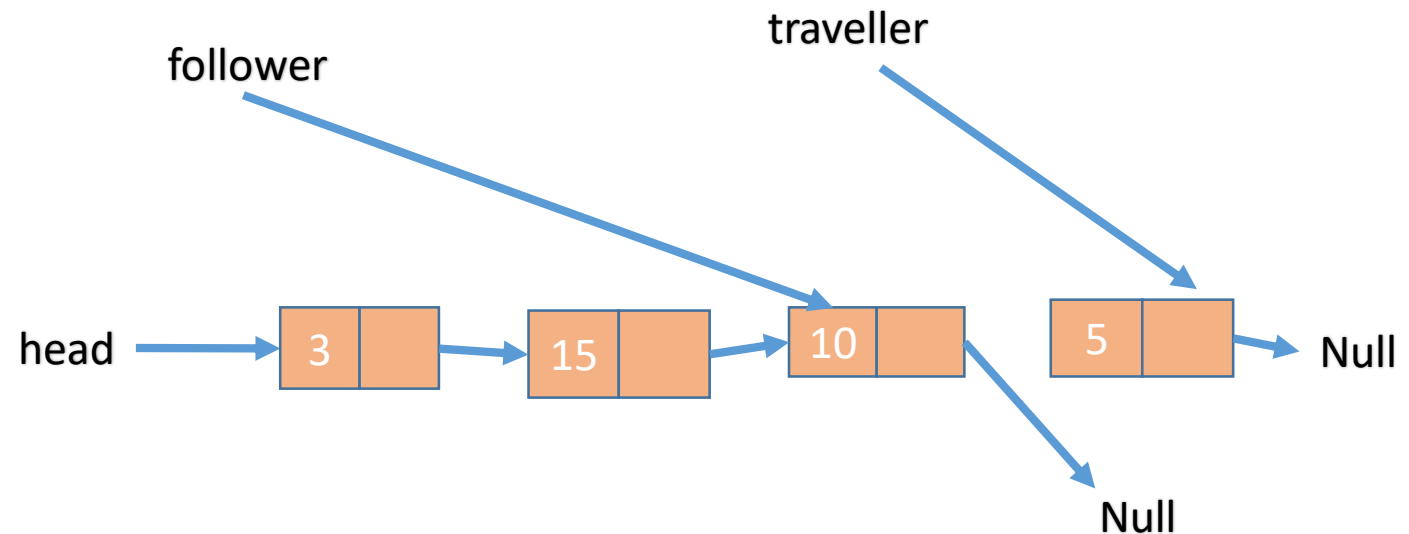
Case 2: The list is not empty
And there are many
Elements in the list



Deletion at the End

```
void deleteTail() {  
    if(head==NULL) {  
        printf("There is no list element to delete\n");  
        return;  
    }  
    struct Node *traveller=head;  
    struct Node *follower=NULL;  
  
    while(traveller->nextNode!=NULL) {  
        follower=traveller;  
        traveller=traveller->nextNode;  
    }  
  
    if(follower!=NULL) {  
        → follower->nextNode=NULL;  
    } else {  
        head=NULL;  
    }  
  
    free(traveller);  
}
```

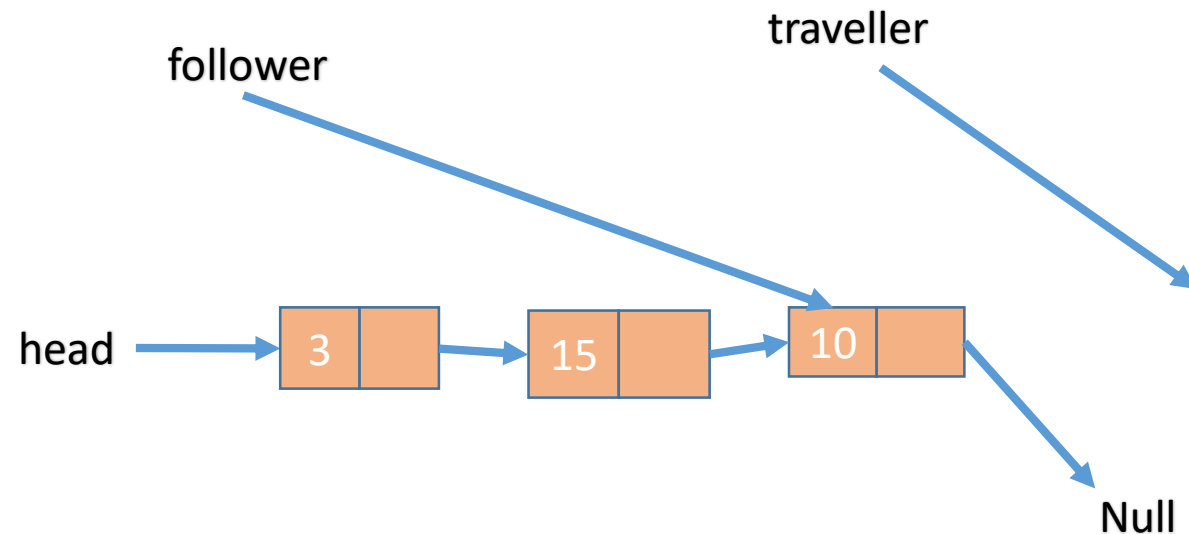
Case 2: The list is not empty
And there are many
Elements in the list



Deletion at the End

```
void deleteTail() {  
    if(head==NULL) {  
        printf("There is no list element to delete\n");  
        return;  
    }  
    struct Node *traveller=head;  
    struct Node *follower=NULL;  
  
    while(traveller->nextNode!=NULL) {  
        follower=traveller;  
        traveller=traveller->nextNode;  
    }  
  
    if(follower!=NULL) {  
        follower->nextNode=NULL;  
    } else {  
        head=NULL;  
    }  
  
    free(traveller);  
}
```

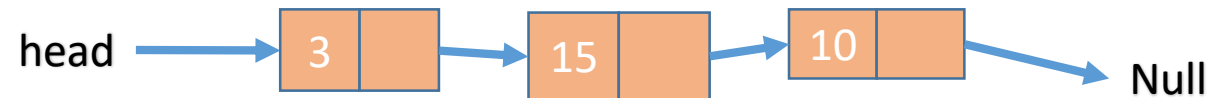
Case 2: The list is not empty
And there are many
Elements in the list



Deletion at the End

```
void deleteTail() {  
    if(head==NULL) {  
        printf("There is no list element to delete\n");  
        return;  
    }  
    struct Node *traveller=head;  
    struct Node *follower=NULL;  
  
    while(traveller->nextNode!=NULL) {  
        follower=traveller;  
        traveller=traveller->nextNode;  
    }  
  
    if(follower!=NULL) {  
        follower->nextNode=NULL;  
    } else {  
        head=NULL;  
    }  
  
    free(traveller);  
}
```

Case 2: The list is not empty
And there are many
Elements in the list



Deletion at the End

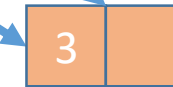
```
void deleteTail() {  
    if(head==NULL) {  
        printf("There is no list element to delete\n");  
        return;  
    }  
    struct Node *traveller=head;  
    struct Node *follower=NULL;  
  
    while(traveller->nextNode!=NULL) {  
        follower=traveller;  
        traveller=traveller->nextNode;  
    }  
  
    if(follower!=NULL) {  
        follower->nextNode=NULL;  
    } else {  
        head=NULL;  
    }  
  
    free(traveller);  
}
```

**Case 3: The list is not empty
And there is only one element**

follower

traveller

head



Null

Deletion at the End

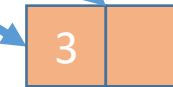
```
void deleteTail() {  
    if(head==NULL) {  
        printf("There is no list element to delete\n");  
        return;  
    }  
    struct Node *traveller=head;  
    struct Node *follower=NULL;  
    while(traveller->nextNode!=NULL) {  
        follower=traveller;  
        traveller=traveller->nextNode;  
    }  
    if(follower!=NULL) {  
        follower->nextNode=NULL;  
    } else {  
        head=NULL;  
    }  
    free(traveller);  
}
```

**Case 3: The list is not empty
And there is only one element**

follower

traveller

head



Null

Deletion at the End

```
void deleteTail() {  
    if(head==NULL) {  
        printf("There is no list element to delete\n");  
        return;  
    }  
    struct Node *traveller=head;  
    struct Node *follower=NULL;  
  
    while(traveller->nextNode!=NULL) {  
        follower=traveller;  
        traveller=traveller->nextNode;  
    }  
    if(follower!=NULL) {  
        follower->nextNode=NULL;  
    } else {  
        head=NULL;  
    }  
  
    free(traveller);  
}
```

**Case 3: The list is not empty
And there is only one element**

follower

traveller

head

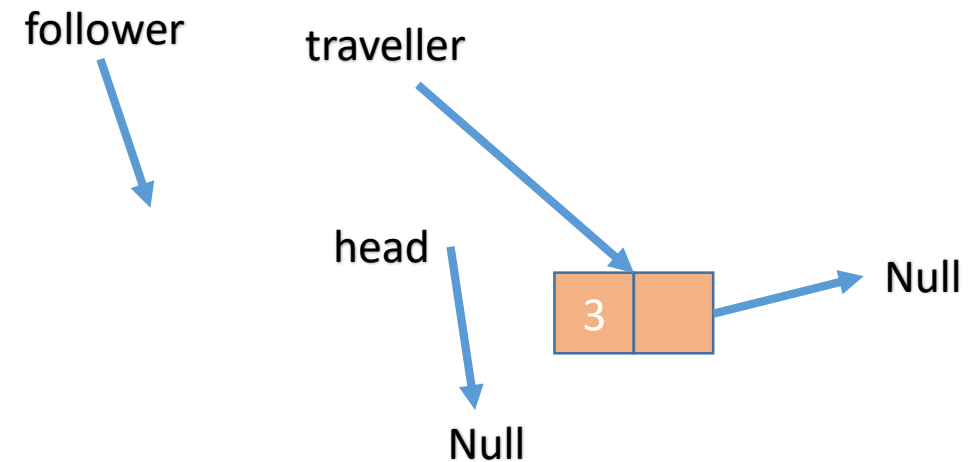
Null



Deletion at the End

```
void deleteTail() {  
    if(head==NULL) {  
        printf("There is no list element to delete\n");  
        return;  
    }  
    struct Node *traveller=head;  
    struct Node *follower=NULL;  
  
    while(traveller->nextNode!=NULL) {  
        follower=traveller;  
        traveller=traveller->nextNode;  
    }  
  
    if(follower!=NULL) {  
        follower->nextNode=NULL;  
    } else {  
        → head=NULL;  
    }  
  
    free(traveller);  
}
```

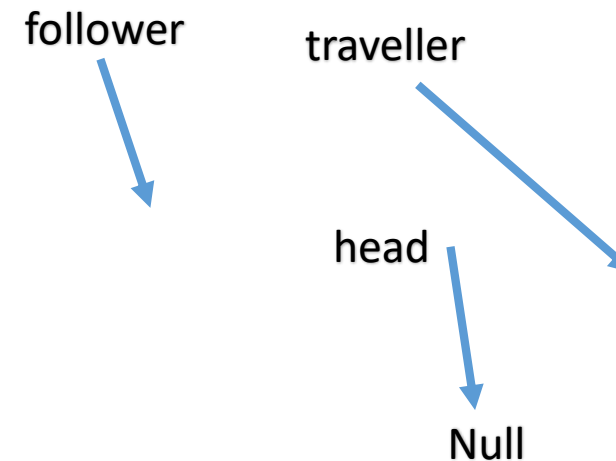
**Case 3: The list is not empty
And there is only one element**



Deletion at the End

```
void deleteTail() {  
    if(head==NULL) {  
        printf("There is no list element to delete\n");  
        return;  
    }  
    struct Node *traveller=head;  
    struct Node *follower=NULL;  
  
    while(traveller->nextNode!=NULL) {  
        follower=traveller;  
        traveller=traveller->nextNode;  
    }  
  
    if(follower!=NULL) {  
        follower->nextNode=NULL;  
    } else {  
        head=NULL;  
    }  
  
    free(traveller);  
}
```

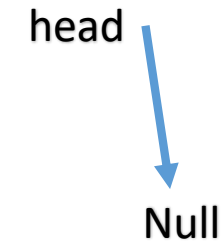
**Case 3: The list is not empty
And there is only one element**



Deletion at the End

```
void deleteTail() {  
    if(head==NULL) {  
        printf("There is no list element to delete\n");  
        return;  
    }  
    struct Node *traveller=head;  
    struct Node *follower=NULL;  
  
    while(traveller->nextNode!=NULL) {  
        follower=traveller;  
        traveller=traveller->nextNode;  
    }  
  
    if(follower!=NULL) {  
        follower->nextNode=NULL;  
    } else {  
        head=NULL;  
    }  
  
    free(traveller);  
}
```

Case 3: The list is not empty
And there is only one element



Deleting a Node with a particular value

```
void deleteAny(int x) {
    if (head == NULL) { //list empty
        printf("List is empty\n");
        return;
    }

    struct Node *cur = head;
    struct Node *prev = NULL;

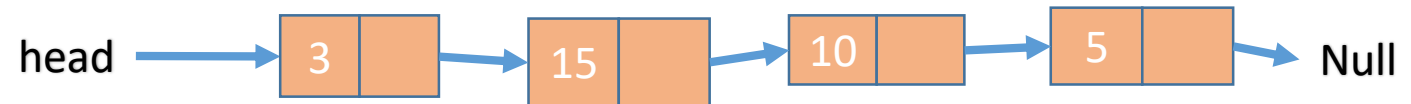
    while (cur != NULL && cur->value != x) {
        prev = cur;
        cur = cur->nextNode;
    }

    if (cur == NULL) { // Value not found in list
        printf("Value not found\n");
        return;
    }

    if (prev != NULL) { // Not the head of the list
        prev->nextNode = cur->nextNode;
    } else { // Node to be deleted is at the head of the list
        head = cur->nextNode;
    }

    free(cur);
    printf("Node deleted successfully\n");
}
```

Case 1: The element is not there
(assume we want to delete 11)



Deleting a Node with a particular value

```
void deleteAny(int x) {
    if (head == NULL) { //list empty
        printf("List is empty\n");
        return;
    }

    struct Node *cur = head;
    struct Node *prev = NULL;

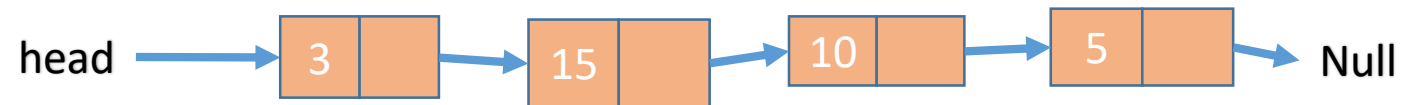
    while (cur != NULL && cur->value != x) {
        prev = cur;
        cur = cur->nextNode;
    }

    if (cur == NULL) { // Value not found in list
        printf("Value not found\n");
        return;
    }

    if (prev != NULL) { // Not the head of the list
        prev->nextNode = cur->nextNode;
    } else { // Node to be deleted is at the head of the list
        head = cur->nextNode;
    }

    free(cur);
    printf("Node deleted successfully\n");
}
```

Case 2: The element is there
(assume we want to delete 10)



Deleting a Node with a particular value

```
void deleteAny(int x) {
    if (head == NULL) { //list empty
        printf("List is empty\n");
        return;
    }

    struct Node *cur = head;
    struct Node *prev = NULL;

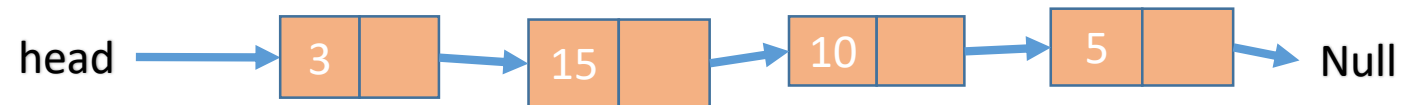
    while (cur != NULL && cur->value != x) {
        prev = cur;
        cur = cur->nextNode;
    }

    if (cur == NULL) { // Value not found in list
        printf("Value not found\n");
        return;
    }

    if (prev != NULL) { // Not the head of the list
        prev->nextNode = cur->nextNode;
    } else { // Node to be deleted is at the head of the list
        head = cur->nextNode;
    }

    free(cur);
    printf("Node deleted successfully\n");
}
```

Case 3: The element is there at the beginning
(assume we want to delete 3)





Thank You